Kubernetes Developer Learning Path

Version 1.0.0, 2026-09-05

Home

Introduction

This course provides a comprehensive overview of the Kubernetes architecture and its powerful extensibility mechanisms. We start by learning how the Kubernetes API functions behind the scenes, exploring how cluster requests are authenticated, authorized, and validated by the control plane. The core of the course focuses on platform automation, teaching you how to extend Kubernetes by defining custom resources and writing custom controllers or operators to automatically manage different workloads. To ensure these extensions are robust, we also cover essential distributed system principles required to build highly available components and safely handle concurrent updates. Finally, the course explores advanced networking and traffic management concepts alongside an exploration of alternative control plane architectures like HyperShift and kcp.

Topics Covered

- Overview — Course roadmap and weekly module previews
- Who is this for? — Target audience and prerequisites
- Week 0 — Familiarizing with Kubernetes — What Kubernetes really is
- Week 1.1 — Kubernetes Resource Model — The declarative intent and kube-apiserver essentials
- Week 1.2 — Controllers, Informers, client-go — The building blocks behind operators
- Week 2.1 — Controllers with controller-runtime — Writing Kube APIs and reconciling with controller-runtime
- Week 2.2 — Going Distributed — Locks, leases, ownership, eventual consistency
- Week 3.1 — Taking More Control — Admissions and custom API servers
- Week 3.2 — The Other Stuff — Networking, security, and cluster hardening
- Week 4.1 — All About kube-scheduler — Scheduling, resources, and DRA
- Week 4.2 — Ubiquitous Topics — Operator landscape and alternative control planes
- Exercises — Hands-on labs for weeks 1, 2, and 3

Prerequisites

This course assumes the following prior experience:

- Familiarity running and managing container workloads within Kubernetes environments
- A basic primer on distributed systems (concurrency, leasing, synchronization)
- A baseline equivalent to holding a CKAD certification or completing Red Hat's DO-180 and DO-280 tracks

See Who is this for? for the full prerequisites and self-assessment checklist.

Reading Lists

- Recommended Reading — Core texts for this course
- Suggested Reading — Supplementary references

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Overview

This course provides a comprehensive overview of the Kubernetes architecture and its powerful extensibility mechanisms. We will start by learning how the Kubernetes API functions behind the scenes, exploring how cluster requests are authenticated, authorized, and validated by the control plane. The core of the course focuses on platform automation, teaching you how to extend Kubernetes by defining custom resources and writing custom controllers or operators to automatically manage different workloads.

Weekly Roadmap

Week 0: Familiarizing with Kubernetes

At its core, Kubernetes is an API-centric, declarative configuration management and container orchestration system. This week orients you with how the API server, etcd, and controllers work together.

Go to Week 0

Week 1.1: Kubernetes Resource Model

A walkthrough on kube-apiserver essentials — the declarative intent of Kubernetes Resources and all the plugin layers (authentication, audit, impersonation, priority & fairness, authorization, admission, storage).

Go to Week 1.1

Week 1.2: Controllers, Informers, and client-go

The basic building blocks behind operators — client-go architecture, Informers, listers, workqueues, caches, and error handling.

Go to Week 1.2

Week 2.1: Controllers with controller-runtime

Writing concrete Kube APIs with controller-runtime — Reconcile loop, kubebuilder, Spec vs Status, Conditions, API validation, CRUD operations, Apply vs Patch vs Update.

Go to Week 2.1

Week 2.2: Going Distributed

Distributed systems challenges — Idempotency, locks, leases, leader election, controller sharding (with OpenShift Ingress Controller case study).

Go to Week 2.2

Week 3.1: Admissions and Beyond the kube-apiserver

From admission control webhooks into writing your own apiserver — ValidatingWebhooks, MutatingWebhooks, policy engines, API discovery, aggregated APIs, custom API servers.

Go to Week 3.1

Week 3.2: The Other Stuff

Networking (Ingress, Gateway API), Service Mesh, AuthN/AuthZ in OpenShift, Security, Cluster Hardening.

Go to Week 3.2

Week 4.1: All About kube-scheduler

kube-scheduler deep dive — scheduling, de-scheduling, resource requests and limits, Dynamic Resource Allocation.

Go to Week 4.1

Week 4.2: Ubiquitous Topics

Operator landscape and alternative control plane architectures — HyperShift, kcp, MicroShift, k3s.

Go to Week 4.2 # Overview

Overview

The Kubernetes Developer Learning Path provides an architectural deep dive into the platform's extensibility and internal mechanics. It is designed for engineers and developers with existing container experience looking to master platform automation and distributed system design.

Key focus areas include:

- **Architecture & Extensibility:** Understanding the Kubernetes control plane, the role of `kube-apiserver`, and the `etcd` persistence layer.
- **Platform Automation:** Building custom resources and operators using the reconcile loop pattern.
- **Core Mechanics:** Deep dives into `client-go` components, including informers, listers, and workqueues.
- **Distributed Systems:** Implementing idempotent state machines, leader election, and optimistic locking via Leases.
- **Advanced Control Plane:** Exploring API aggregation, admission control webhooks, and modern traffic management paradigms like the Gateway API and Service Mesh.
- **Security & Operations:** Examining AuthN/AuthZ models, cluster hardening techniques, and alternative control plane form factors.

Overview

Kubernetes architecture overview and extensibility mechanisms

Kubernetes Architecture Overview and Extensibility Mechanisms

Kubernetes is far more than a container orchestrator; it is a **distributed operating system for the cloud**. At its core, it functions as a highly available, API-centric, declarative configuration management system. Understanding its architecture is the first step toward mastering the extensibility that makes Kubernetes the industry standard for platform engineering.

The Control Plane: The Brain of the Cluster

The Kubernetes control plane is the central nervous system of the cluster. It makes global decisions about the cluster (e.g., scheduling), detects and responds to cluster events, and maintains the desired state.

- **API Server (`kube-apiserver`):** The gateway to the cluster. All internal and external components communicate through the API server. It exposes the Kubernetes API and serves as the front end for the control plane.
- **etcd:** The consistent and highly-available key-value store used as Kubernetes' backing store for all cluster data.
- **Scheduler (`kube-scheduler`):** Watches for newly created Pods with no assigned node and selects a node for them to run on.
- **Controller Manager (`kube-controller-manager`):** Runs controller processes, which regulate the state of the cluster by reconciling the current state toward the desired state.

The API-Centric Lifecycle

The `kube-apiserver` processes requests through a sophisticated plugin chain. When a user or controller submits a resource (e.g., a `Deployment` or `CustomResourceDefinition`), the request flows through:

1. **Authentication:** Who are you? (e.g., X.509 certs, OIDC).
2. **Audit:** Logging the request for security compliance.
3. **Impersonation:** Checking if the user is authorized to act as another user.
4. **Authorization:** Are you allowed to perform this action? (RBAC).
5. **Admission Control:** Mutating or Validating the resource against business logic before it is persisted.
6. **Storage:** Persistence in `etcd`.

Extensibility: Defining Your Own Reality

The true power of Kubernetes lies in its ability to be extended. You are not limited to the standard resources (Pods, Services, Deployments). You can extend the API to manage your own domain-specific abstractions.

1. Custom Resource Definitions (CRDs)

CRDs allow you to define custom objects in the Kubernetes API. Once a CRD is registered, the API server handles the lifecycle of these custom resources just as it does for native resources.

2. The Controller Pattern

The controller pattern is the mechanism that makes Kubernetes "act." Controllers watch the API server for changes to a specific resource type. When an event occurs, the controller performs a **Reconciliation Loop**:

- **Observe:** Get the current state of the resource.
- **Analyze:** Compare the current state against the desired state defined in the resource `spec`.
- **Act:** Perform operations (e.g., creating Pods, updating status) to close the gap between the two states.

3. Admission Webhooks

While controllers handle "post-persistence" logic, admission webhooks allow you to intercept requests **before** they are persisted to `etcd`. * **Validating Webhooks:** Used to reject a request if it violates your security policies (e.g., "Only allow images from the internal registry"). * **Mutating Webhooks:** Used to inject default values or modify the resource content (e.g., automatically adding sidecars).

Why Extensibility Matters

By leveraging the native Kubernetes control plane patterns, developers can build **Operators**—software that encodes human operational knowledge into code. This allows for: * **Automation:** Managing complex, stateful applications (databases, message queues) as if they were native Kubernetes objects. * **Consistency:** Ensuring every team deploys resources following the same organizational standards. * **Resiliency:** Utilizing the Kubernetes reconciliation loop to ensure systems are self-healing and continuously match the defined desired state.

In the next modules, we will dive deeper into `client-go` and `controller-runtime` to build these patterns from the ground up, moving from simple API interactions to complex, distributed reconciliation logic.

Control plane authentication, authorization, and validation

Control Plane Authentication, Authorization, and Validation

In the Kubernetes architecture, the `kube-apiserver` acts as the central gatekeeper. Before any resource is persisted to `etcd`, the request must pass through a multi-layered security pipeline. Understanding this flow is critical for developers building custom operators or extending the platform.

The API Request Lifecycle

When a user or a service account interacts with the Kubernetes API, the `kube-apiserver` processes the request through three distinct phases:

1. **Authentication (AuthN):** Determining **who** is making the request.
2. **Authorization (AuthZ):** Determining **what** the user is allowed to do.
3. **Admission Control:** Validating or mutating the request content before it is finalized.

---

1. Authentication (AuthN)

The API server relies on one or more authentication modules to verify the caller's identity. If a request cannot be authenticated, the API server rejects it with an HTTP 401 response.

- **X.509 Client Certs:** Managed by passing a CA file to the API server.
- **Static Token Files:** Simple, but generally discouraged for production.
- **OpenID Connect (OIDC):** Integration with external Identity Providers (IdPs) like Dex or Keycloak.
- **Service Account Tokens:** Standard for internal cluster communication.

2. Authorization (AuthZ)

Once the identity is established, the request must be authorized. Kubernetes supports several modes, which are checked in order:

- **RBAC (Role-Based Access Control):** The standard mechanism using `Roles`/`ClusterRoles` and `RoleBindings`/`ClusterRoleBindings` to define permissions based on actions (verbs) and resources.
- **ABAC (Attribute-Based Access Control):** Policies defined via JSON files.
- **Webhook:** The API server delegates the authorization decision to a remote HTTP service.

Note: In managed environments like OpenShift, AuthZ is tightly integrated with the cluster's identity provider, often mapping OIDC groups to internal RBAC roles.

3. Admission Control (Validation & Mutation)

This is the final phase of the lifecycle. Admission Controllers act as plugins that intercept requests after they are authenticated and authorized, but before the object is persisted to `etcd`.

Mutating Admission Controllers

These controllers can modify the object content before it is stored. * **Use case:** Injecting sidecars, setting default values for security contexts, or adding default labels.

Validating Admission Controllers

These controllers check the request against defined policies. If the validation fails, the request is rejected. * **Use case:** Ensuring that container images come from an approved registry, requiring specific labels on Namespaces, or enforcing resource limit ranges.

Practical Perspective: Admission Webhooks

While built-in controllers cover standard scenarios, developers often need to implement custom logic. This is achieved through **Dynamic Admission Webhooks**:

Type	Behavior
`MutatingAdmissionWebhook`	Modifies the object (e.g., adding `imagePullSecrets`).
`ValidatingAdmissionWebhook`	Permits or denies the object based on criteria (e.g., "Must have `owner` label").

Designing for Security

To ensure a robust control plane: * **Fail-closed:** Always configure webhooks to fail closed (deny the request) if the webhook service is unreachable. * **Idempotency:** Admission logic must be idempotent; the same request processed twice should yield the same result without side effects. * **Latency Impact:** Admission webhooks are in the critical path of the API request. Keep logic lightweight to avoid slowing down `kubectl apply` operations.

Summary Checklist for Developers

- ▢ **Identify:** Are you interacting as a user or a ServiceAccount?
- ▢ **Permission:** Does your RBAC definition follow the principle of least privilege?
- ▢ **Validation:** Is your custom resource validation logic handled via CEL (Common Expression Language) or a custom Validating Webhook?
- ▢ **Audit:** Ensure your control plane is configured to output logs so you can audit which identities performed which actions.

Platform automation through custom resources and operators

Platform Automation: Custom Resources and Operators

Overview

Kubernetes is more than a container orchestrator; it is a platform for building platforms. The primary mechanism for this extensibility is the **Kubernetes Resource Model (KRM)**, which allows developers to extend the API to manage domain-specific logic. By leveraging **Custom Resource Definitions (CRDs)** and **Controllers**, you can codify operational knowledge into software, creating what we commonly call an **Operator**.

Custom Resources: Extending the API

A **Custom Resource (CR)** is an extension of the Kubernetes API that allows you to store and retrieve structured data. When you define a CRD, the `kube-apiserver` creates a new RESTful endpoint for your resource.

- **Spec**: Defines the **desired state**—what the user wants the system to look like.
- **Status**: Reflects the **observed state**—what the system currently looks like.

Avoid putting transient data in the `Spec`. The `Spec` should contain configuration that persists even if the controller is restarted.

The Operator Pattern: Automating Operations

An Operator is a software extension that uses custom resources to manage applications and their components. It follows the **Asynchronous Controller Reconciliation Pattern**:

1. **Watch**: The controller registers informers to monitor events on specific resources.
2. **Diff**: When an event occurs, the controller compares the `Spec` (intent) against the actual state of the system.
3. **Act**: The controller executes a series of actions (CRUD operations) to move the system toward the desired state.
4. **Reconcile**: This loop runs continuously, ensuring the system remains in the desired state despite external changes.

Why Custom Controllers?

While `kubectl` and manual manifests work for simple workloads, complex systems require: * **Encapsulation**: Bundling complex installation and configuration logic. * **Self-Healing**: Automatic recovery from failures without human intervention. * **Consistency**: Ensuring every instance of an application is deployed with the same parameters.

Hands-on Lab: Scaffolding a Controller with Kubebuilder

In this lab, we will use `kubebuilder` to generate the boilerplate for a custom operator.

Prerequisites

- Go (latest version)
- `kubectl`

- `kubebuilder` CLI
- A running Kubernetes cluster (e.g., Minikube or Kind)

Step 1: Initialize the Project

Create a new directory and initialize your project:

```
mkdir my-operator
cd my-operator
kubebuilder init --domain example.com --repo github.com/example/my-operator
```

Step 2: Create the API (CRD)

Generate a new API group for your resource (e.g., `AppService`):

```
kubebuilder create api --group webapp --version v1 --kind AppService
```

Step 3: Define the Spec

Edit `api/v1/appservice_types.go` to define the schema:

```
type AppServiceSpec struct {
    // Size is the desired number of replicas
    Size int32 `json:"size,omitempty"`
}

type AppServiceStatus struct {
    // Nodes are the names of the pods in the deployment
    Nodes []string `json:"nodes,omitempty"`
}
```

Step 4: Implement the Reconcile Loop

Locate `internal/controller/appservice_controller.go` and implement the `Reconcile` function. Your logic should check if the resource exists, create child resources if missing, and update the `Status`.

```
func (r *AppServiceReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    // 1. Fetch the AppService instance
    // 2. Logic to ensure deployment size matches Spec.Size
    // 3. Update Status
    return ctrl.Result{}, nil
}
```

Step 5: Run the Controller

```
make install  # Install CRDs into the cluster
make run      # Run the controller locally
```

Key Takeaways

- **Declarative Intent**: The `Spec` represents the user's intent, not a command sequence.
- **Idempotency**: The reconcile loop must be idempotent—running it twice must produce the same result as running it once.
- **Event-Driven**: Controllers rely on `client-go` informers to efficiently listen for changes rather than polling the API server.

Distributed system principles for high availability

Distributed System Principles for High Availability in Kubernetes

In a distributed system like Kubernetes, high availability (HA) is not merely about keeping processes running; it is about ensuring the system maintains a consistent state despite network partitions, hardware failures, or concurrent updates. When extending Kubernetes through custom controllers, you are essentially building a distributed state machine.

The Challenge of Distributed State

Kubernetes operates on the principle of **eventual consistency**. The control plane does not guarantee that a cluster will be in a specific state at a specific microsecond. Instead, it guarantees that, given enough time, the cluster will converge toward the "desired state" defined in your objects.

When multiple controller instances manage the same resources, you face two primary challenges: 1. **Race Conditions:** Multiple controllers attempting to modify the same resource simultaneously. 2. **Split-Brain Scenarios:** Different controller instances making conflicting decisions, leading to state corruption or service instability.

Foundational Principles for HA Controllers

To build robust, highly available automation, your controllers must adhere to these distributed systems principles:

1. Idempotency

An operation is idempotent if performing it multiple times yields the same result as performing it once. In Kubernetes, your `Reconcile` function must be designed to be idempotent. * **Design Pattern:** Always read the current state (Status/Live state) before acting. If the resource is already in the desired state, do nothing. * **Why it matters:** Controllers can be triggered by spurious events or retries. Non-idempotent operations often lead to resource exhaustion or duplicate API calls.

2. Optimistic Concurrency Control (OCC)

Kubernetes uses `resourceVersion` in metadata to implement OCC. When you update an object, the API server checks if the `resourceVersion` you provided matches the current version in `etcd`. * **Mechanism:** If a conflict occurs (409 Conflict), your controller must handle the error by re-fetching the latest version of the object and re-running the reconciliation logic.

3. Leader Election

In high-availability deployments, you often run multiple replicas of a controller (e.g., via a `Deployment`). However, you usually only want **one** active controller to perform reconciliation at any given time to avoid conflicting actions. * **Lease Objects:** Kubernetes uses the `coordination.k8s.io/v1 Lease` API. Controllers compete for a lock on a Lease resource. The winner becomes the leader, while others wait in a standby state.

4. Controller Sharding

When a single controller cannot handle the sheer volume of resources, you must employ **sharding**. This is the process of partitioning the workload across multiple controller instances based on a specific key (e.g., namespace or a custom label). * **Case Study (OpenShift Ingress):** The ingress controller shards the routing table. Each controller instance is responsible for a subset of the total routes, ensuring that the control plane remains responsive as the number of ingress objects grows.

Distributed Sync Workflow: Hands-On

To implement high availability in your own controllers, you must move beyond a simple loop.

Example 1. Hands-on Lab: Implementing Leader Election with Leases

In this lab, you will configure your `client-go` controller to respect leader election using the `leaderelection` package.

1. **Initialize the Leader Election Config:** Define a `LeaderElectionConfig` that specifies the `Lease` name and the namespace where the lock will be held.

```
[source,go]
----
import "k8s.io/client-go/tools/leaderelection"
```

```
// Define the lock
lock := &resourcelock.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{Name: "my-controller-lock", Namespace: "kube-
system"},
    Client:    client.CoordinationV1(),
    LockConfig: resourcelock.ResourceLockConfig{Identity: podName},
}
```

```
----
```

2. **Run the Election Loop:** Wrap your main reconciliation logic inside the `leaderelection.RunOrDie` function. This ensures your code only executes when the lease is acquired.

```
[source,go]
----
leaderelection.RunOrDie(context.TODO(), leaderelection.LeaderElectionConfig{
    Lock: lock,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            // Start your Informers and Reconcile loop here
        },
        OnStoppedLeading: func() {
            log.Fatalf("Lost leader lock")
        },
    },
})
----
```

3. **Verification:** Deploy two replicas of your controller. Observe that only one log entry indicates "Leading", and if you delete that pod, the second replica automatically picks up the lease.

Summary for Architectural Success

- **Design for failure:** Always assume your controller might restart or encounter a network partition.
- **Leverage the API Server:** Never store "state" inside your controller's memory. Always persist state in the Kubernetes API using Status fields or Custom Resources.
- **Reconcile, don't trigger:** Your logic should be driven by the difference between current state and desired state, not by a sequence of events.

Advanced networking, traffic management, and control plane architectures

Advanced Networking, Traffic Management, and Control Plane Architectures

Overview

As Kubernetes clusters grow in scale and complexity, standard Service and Ingress primitives often prove insufficient for enterprise-grade requirements. Advanced networking focuses on decoupling traffic routing from underlying infrastructure, while diverse control plane architectures address

the needs of edge computing, multi-tenancy, and high-density environments.

1. Advanced Networking and Traffic Management

Modern Kubernetes networking has evolved from simple L4 load balancing to sophisticated L7 traffic management.

Gateway API vs. Ingress

While the standard `Ingress` resource provides a basic way to expose HTTP/S routes, it suffers from fragmentation across different ingress controller implementations (e.g., NGINX, HAProxy, Contour). The **Gateway API** addresses this by: * **Role-Oriented Design:** Separating infrastructure ownership (GatewayClasses) from application routing (HTTPRoutes). * **Expressiveness:** Providing native support for advanced traffic patterns like header-based routing, traffic splitting, and weighted routing. * **Extensibility:** Allowing custom policies to be attached directly to routes.

Service Mesh Concepts

When networking requirements exceed simple routing—such as mTLS, advanced circuit breaking, or fine-grained observability—a **Service Mesh** (e.g., Istio, Linkerd) is typically employed. * **Control Plane:** Orchestrates configuration and security policies. * **Data Plane:** Sidecar proxies (like Envoy) intercept all network traffic to provide transparent encryption, retries, and telemetry collection without modifying application code.

2. Alternative Control Plane Architectures

The "standard" Kubernetes control plane (kube-apiserver, etcd, scheduler, controller-manager) is optimized for general-purpose clusters. However, varying use cases have necessitated the emergence of alternative architectures.

HyperShift (Hosted Control Planes)

HyperShift decouples the control plane from the worker nodes. By running control planes as pods inside a management cluster, you gain: * **Rapid Provisioning:** Faster deployment of clusters. * **Resource Efficiency:** Reduced overhead by sharing management cluster infrastructure. * **Isolation:** Control planes are isolated, making them easier to upgrade and manage independently.

kcp: Kubernetes-like API Server

`kcp` is designed for scenarios where a single API surface is needed to manage resources across thousands of clusters. It provides a "Kubernetes-like" experience without forcing the overhead of a full cluster control plane for every tenant. It focuses on: * **Logical Clusters:** Enabling multi-tenancy at the API layer. * **API Agnosticism:** Allowing users to define custom resources that aren’t bound to a specific runtime cluster.

Edge and Lightweight Form Factors

For constrained environments, standard Kubernetes is too heavy. Solutions like **MicroShift** (the edge-optimized distribution of OpenShift) and **k3s** minimize the footprint by: * **Consolidation:** Combining control plane components into a single binary. * **Storage Optimization:** Replacing etcd with lighter key-value stores (e.g., SQLite) in some configurations. * **Aggressive Pruning:** Removing

non-essential features and legacy drivers.

Summary of Architectural Trade-offs

| Architecture | Primary Use Case | Scaling Approach | | :--- | :--- | :--- | | **Standard K8s** | General purpose, Cloud-native | Horizontal node scaling | | **HyperShift** | Multi-tenant, Managed Services | Control Plane consolidation | | **kcp** | Global-scale API management | Logical clustering | | **MicroShift/k3s** | Edge, IoT, Industrial | Resource minimization |

Further Exploration

- **Design Considerations:** When choosing a control plane architecture, consider the "blast radius"—how failures in the control plane propagate to the managed workloads.
- **Networking Security:** Always prioritize mTLS for service-to-service communication when moving to advanced traffic management solutions to ensure zero-trust compliance.

Who Is This For? Prerequisites

Target Audience

This learning path is designed for engineers, system administrators, and developers who already possess practical experience working with containerized environments and Kubernetes. If you have deployed applications on Kubernetes and want to move beyond basic usage into platform-level development, this course is for you.

Recommended Prerequisites

A foundational understanding of distributed systems principles is strongly recommended before beginning this course. Familiarity with concepts such as concurrency, leasing, and synchronization will help you get the most out of the material on controllers, leader election, and eventual consistency.

Ideal Baseline

The ideal starting point is proficiency equivalent to one of the following:

- **Certified Kubernetes Application Developer (CKAD)** certification
- Completion of Red Hat **DO-180** (Introduction to Containers, Kubernetes, and Red Hat OpenShift) and **DO-280** (Red Hat OpenShift Administration) training tracks

Either path ensures you have the hands-on Kubernetes experience needed to engage with the advanced topics in this course.

Self-Assessment Checklist

Before starting, review the checklist below. You should be comfortable with most of these items:

- ☐ You can create and manage Deployments, Services, and ConfigMaps
- ☐ You understand container image building and registries
- ☐ You are comfortable reading and writing YAML manifests
- ☐ You have basic familiarity with Go programming
- ☐ You can use `kubectl` to inspect and debug cluster resources
- ☐ You understand the difference between a Pod, a ReplicaSet, and a Deployment
- ☐ You have worked with namespaces and resource quotas
- ☐ You are familiar with basic RBAC concepts (Roles, RoleBindings) # Who is this for? pre-requisites.

Who is this for? Pre-requisites

This learning path is designed for engineers, system administrators, and developers who possess practical experience with containerized environments. To ensure success, participants should have a foundational understanding of distributed systems.

- **Target Audience:** Professionals working with container technology and infrastructure automation.
- **Recommended Prerequisites:** Basic knowledge of distributed system principles.
- **Ideal Baseline:** Completion of CKAD certification, or Red Hat DO-180 and DO-280 tracks.

Who Is This For? Prerequisites

Target audience: Engineers, sys admins, and developers with container experience

Target Audience and Prerequisites

This learning path is designed for technical professionals who have moved beyond basic container usage and are ready to master the internal mechanics, extensibility, and architectural patterns of Kubernetes.

Who is this for?

This content is curated for:

- **Software Engineers:** Developers who need to extend Kubernetes functionality by building custom controllers, operators, or specialized API servers.
- **Systems Administrators (SREs/Platform Engineers):** Professionals responsible for managing, scaling, and hardening complex Kubernetes environments.
- **Platform Architects:** Those tasked with designing automated workflows or deploying alternative control plane architectures (e.g., HyperShift, kcp).

Recommended Prerequisites

To derive maximum value from this path, you should possess a solid foundation in container orchestration. We assume you are comfortable with:

- **Container Fundamentals:** You understand images, container lifecycles, and basic networking (e.g., `podman` or `docker` workflows).
- **Kubernetes Basics:** You are familiar with standard resources (Deployments, Services, ConfigMaps) and the `kubectl` CLI.
- **Distributed Systems Concepts:** You have a working knowledge of asynchronous communication, state consistency, and basic concurrency models.
- **Programming Literacy:** Because we will be interacting with `client-go` and implementing controllers, proficiency in **Go (Golang)** is highly recommended.

Ideal Baseline

We recommend that participants meet one of the following benchmarks to ensure a successful learning experience:

Benchmark	Description
CKAD Certification	Demonstrates hands-on proficiency in designing, building, and deploying cloud-native applications on Kubernetes.

Benchmark	Description
Red Hat DO-180	Familiarity with Introduction to Containers, Kubernetes, and Red Hat OpenShift.
Red Hat DO-280	Proficiency in OpenShift Administration: Core concepts of managing enterprise-grade Kubernetes clusters.

If you are familiar with the declarative nature of Kubernetes but are new to the internal `kube-apiserver` request/response lifecycle, we will cover these foundational concepts in the upcoming modules. If your Go programming skills are rusty, we suggest reviewing the Go Tutorial before starting the **client-go** lab exercises.

Why this path matters

Beyond managing static workloads, this curriculum moves you toward **Platform Engineering**. By the end of this path, you will transition from a consumer of Kubernetes resources to an architect capable of:

1. Automating infrastructure through **Custom Resources (CRDs)**.
2. Building resilient **Controllers** that handle complex state reconciliation.
3. Securing the cluster via **Admission Webhooks**.
4. Understanding the distributed systems "magic" that keeps the Control Plane stable under high load.

Recommended prerequisites: Basic distributed systems knowledge

Prerequisites: Preparing for the Kubernetes Developer Learning Path

To succeed in this advanced Kubernetes development track, it is essential to have a solid foundation. While this course focuses on the internal mechanics of Kubernetes extensibility—such as custom controllers, admission webhooks, and distributed state management—these concepts rely heavily on the principles of distributed systems.

Who is this for?

This curriculum is designed for **Software Engineers, Site Reliability Engineers (SREs), and Platform Architects** who have moved beyond simply deploying applications and are now tasked with extending the Kubernetes control plane itself.

Recommended Prerequisites

Before beginning this module, you should be comfortable with the following competencies:

Category	Requirement
Containerization	Proficiency in container lifecycles, OCI image standards, and resource management (Requests/Limits). You should be familiar with the DO-180 (Introduction to Containers, Kubernetes, and Red Hat OpenShift) level of knowledge.
Kubernetes Administration	Familiarity with standard cluster operations. You should be comfortable with `kubectl`, the structure of common resources (Pods, Deployments, Services), and standard RBAC (Role-Based Access Control) configuration (equivalent to DO-280 or CKAD certification).
Programming	Strong proficiency in **Go (Golang)** is highly recommended. As the Kubernetes codebase is primarily written in Go, and `client-go` is the primary library for interaction, comfort with Go interfaces, concurrency primitives (goroutines/channels), and error handling is vital.

Distributed Systems Fundamentals

Because we will be building controllers that interact with a highly available, eventually consistent system, you should have a baseline understanding of:

- **Eventual Consistency:** Understanding that the state of a cluster is a distributed consensus problem. You should understand how the control plane moves from a "desired state" to an "actual state."
- **Concurrency and Race Conditions:** Knowledge of how to handle concurrent modifications to resources, which is critical when writing controllers that may be deployed in high-availability (HA) modes with multiple replicas.
- **API Semantics:** Understanding RESTful interactions, HTTP status codes, and the difference between synchronous (blocking) and asynchronous (event-driven) processing.

Why these prerequisites matter

In this course, we move away from "using" the cluster to "orchestrating" it. When you build a custom operator, you are essentially writing code that operates as part of the Kubernetes control plane. If your controller logic is not designed with distributed systems principles (such as idempotency and proper error handling), you risk causing "flapping" states or system-wide instability.

If you feel your knowledge in these areas is limited, we suggest reviewing: * **Designing Distributed Systems** by Brendan Burns (O'Reilly). * The official Kubernetes documentation on the [Reference Architecture](https://kubernetes.io/docs/concepts/architecture/).

Ideal baseline: CKAD certification, or Red Hat DO-180 and DO-280 tracks

Preparing for the Kubernetes Developer Learning Path

To succeed in this curriculum, you must possess a solid foundation in container orchestration. This learning path is designed for engineers, system administrators, and developers who are ready to transition from **using** Kubernetes to **extending** it.

The Ideal Baseline

We expect participants to have a working understanding of the Kubernetes ecosystem. Our curriculum assumes you are comfortable with declarative configuration and the standard resource lifecycle. You should be prepared to meet one of the following benchmarks:

- **CKAD (Certified Kubernetes Application Developer):** This certification confirms your ability to design, build, and deploy cloud-native applications. If you hold this certification, you already have the necessary skills in handling Pods, Deployments, Services, and ConfigMaps—the fundamental building blocks we will manipulate via custom controllers.
- **Red Hat DO-180 (Introduction to Containers, Kubernetes, and Red Hat OpenShift):** This track provides the essential vocabulary for containerized environments, image management, and basic orchestration.
- **Red Hat DO-280 (Red Hat OpenShift Administration II: Operating a Production Kubernetes Cluster):** This track dives deeper into the operational aspects of the platform, providing the necessary context regarding how clusters are secured, networked, and maintained—all of which are critical when writing controllers that interact with the cluster's control plane.

If you do not hold these specific credentials, ensure you have equivalent practical experience. You should be familiar with: * The `kubectl` command-line interface and basic API interaction. * YAML-based manifest management. * The fundamental concepts of distributed systems (e.g., eventual consistency, API-centric communication).

Why these prerequisites matter

The content we will cover—ranging from `client-go` informers to Custom Resource Definitions (CRDs) and admission webhooks—builds directly upon the core Kubernetes resource model.

If you are unfamiliar with how the `kube-apiserver` processes a request or how a deployment maintains its state through reconciliation, the jump into `client-go` architecture and controller reconciliation patterns will be significantly more difficult. By starting with the proficiency level defined by the **CKAD** or **DO-180/DO-280** tracks, you ensure that you can focus on the **extension** mechanisms of Kubernetes rather than struggling with basic application deployment patterns.

If you feel your knowledge in these areas is rusty, revisit the official Kubernetes documentation on **Declarative Management** and the **Kubernetes API** before beginning the hands-on labs in this learning path.

Week 0 — Familiarizing with Kubernetes

Welcome to the orientation week of the Kubernetes Developer Learning Path. In this module we define what Kubernetes really is at its core — an API-centric, declarative configuration management and container orchestration system. We also introduce the control plane architecture at a high level, covering the components that keep a cluster running. Detailed treatments of the Kubernetes resource model and controller internals follow in Week 1.1 and Week 1.2, respectively.

What Kubernetes Really Is

At its core, Kubernetes is an API-centric, declarative configuration management and container orchestration system. Rather than issuing imperative commands to tell the server what to do, you interact with the system by declaring the desired state of your resources as pure data.

The heart of the control plane is the API server, which acts as a central coordination point and persists this desired state into a key-value store called etcd.

To make reality match those declarations, Kubernetes relies on an ecosystem of asynchronous controllers. These controllers continuously watch the API server for changes, compare the current observed state of the world to your desired state, and actively reconcile any differences.

This separation of responsibilities — where the API server maintains shared state and independent controllers actuate it — is the same fundamental architecture you will use and extend when building your own custom operators.

A Three-Layer Mental Model

It helps to think of any declarative configuration management system — Kubernetes included — as composed of three distinct layers:

- **Layer 1 — Resource Schema:** The definition of which types of resources exist and what shape they take. In Kubernetes, this is expressed through API group-version-kind (GVK) definitions, Protocol Buffer schemas for built-in types, and OpenAPI v3 validation schemas for custom resources.
- **Layer 2 — Concrete Resource Syntax:** The raw YAML or JSON data that users author and submit to the API server. Kubernetes deliberately accepts only fully-realized resource definitions, pushing abstraction (templating, parameterization) to external tools such as Helm, Kustomize, or general-purpose programming languages.
- **Layer 3 — Convergence Engine:** The controllers that take a concrete set of resource declarations and actuate the managed system to match. Kubernetes has perhaps the most explicit notion of this layer of any declarative system, embodied in its controller architecture.

This layered model clarifies why Kubernetes separates concerns the way it does: the API server owns Layers 1 and 2 (schema and storage of declared state), while controllers independently implement Layer 3 (making reality converge). The full treatment of the resource model — including how schemas, serialization, and storage work together — is covered in Week 1.1. Controller internals and the client-go machinery that powers reconciliation are explored in Week 1.2.

Control Plane Architecture Primer

The Kubernetes control plane is the set of components responsible for maintaining the desired state of your cluster. Here is a brief overview of each core component.

Core Components

kube-apiserver

The central coordination point. All cluster communication — whether from users, controllers, or the kubelet — flows through the API server. It exposes a RESTful API, handles authentication, authorization, and admission control, and is the only component that communicates directly with etcd.

etcd

A distributed key-value store that holds the entire desired state of the cluster. It uses the Raft consensus algorithm to maintain consistency across replicas. For performance and consistency reasons, etcd typically scales vertically (more CPU and RAM) rather than horizontally; expanding beyond three or five members introduces consensus overhead.

kube-scheduler

Watches for newly created Pods that have no assigned node and selects a suitable node for each one. Scheduling decisions take into account resource requirements, affinity and anti-affinity rules, taints, tolerations, and other constraints.

kube-controller-manager

Runs a collection of built-in controllers — including the Deployment controller, ReplicaSet controller, Job controller, and many others — that continuously reconcile the current state of the cluster toward the desired state declared in the API server.

On an OpenShift cluster, additional API servers (such as the OpenShift API server) run alongside kube-apiserver, registered via the API aggregation layer. These extensions add OpenShift-specific resource types while preserving the same architectural pattern.

The API Request Lifecycle

Every request that reaches the API server passes through a well-defined pipeline:

1. **Authentication** — Verify the identity of the caller.
2. **Authorization** — Determine whether the authenticated identity is permitted to perform the requested action.
3. **Admission Control** — Mutating and validating admission webhooks inspect, modify, or reject the request.
4. **Storage** — Upon successful admission, the resource is persisted to etcd.

This lifecycle is the gateway through which all cluster state changes flow. You will walk through each stage in detail in Week 1.1.

Week 1.1 — Kubernetes Resource Model

This week you will explore how the Kubernetes API works behind the scenes — not just what you can do with it, but why it is designed the way it is. We begin with the Kubernetes Resource Model document, which lays out the assumptions, principles, and conventions that make the API composable and extensible. From there, you will walk through the kube-apiserver request pipeline to understand the stages every API call passes through before state is persisted to etcd. Finally, we examine a three-layer model of declarative configuration management that frames Kubernetes as a system with unusually strict separation between resource schemas, concrete syntax, and convergence engines. Together, these readings establish the mental model you will rely on throughout the rest of this learning path — from writing controllers in Week 1.2, to building admission webhooks in Week 3.1.

The Kubernetes Resource Model

Kubernetes is API-Centric

Kubernetes is not just API-driven — it is **API-centric**. At the center of the control plane is the apiserver, which implements common functionality for all of the system's APIs. Both user clients and components implementing business logic (called controllers) interact through the same APIs. All persistent cluster state is stored in one or more instances of etcd.

This means the distinction between being part of the system and being built on top of the system is deliberately blurred. With the growth of API extension mechanisms such as Custom Resource Definitions (CRDs) and aggregated API servers, the number of resource types can be extended without modifying Kubernetes itself. Any API using the same mechanisms and patterns automatically works with libraries and tools that already integrate support for the model — making integration O(M) work for M tools rather than O(NM) for N APIs and M tools.

Declarative Control

Kubernetes supports declarative control by recording user intent as the desired state in its API resources. The intent is carried out by asynchronous controllers, which interact through the Kubernetes API — they do not access etcd directly, nor do they communicate via private internal APIs.

Controllers continuously strive to make the observed state match the desired state and report back their status to the apiserver asynchronously. All state — desired and observed — is visible through the API to users and to other controllers. The API resources serve as coordination points, common intermediate representation, and shared state.

Controllers are **level-based** rather than edge-based to maximize fault tolerance. This means a controller operates correctly given just the desired state and the observed state, regardless of how many intermediate state updates may have been missed. However, controllers can achieve the benefits of an edge-triggered implementation by monitoring changes via the watch API, which minimizes reaction latency and redundant work.

Resource Model Properties

The Kubernetes resource model encodes several properties that reinforce the system's resilience and extensibility:

- **No hidden internal APIs** — all APIs are visible and available, subject only to authorization policy. There are few direct inter-component APIs. To handle complex use cases, one can access lower-level APIs in a fully transparent manner or add new APIs as necessary.
- **No atomic cross-resource transactions** — desired state is updated immediately but actuated asynchronously and eventually. Kubernetes does not support atomic transactions across multiple resources, pessimistic locking, or synchronous success preconditions based on actuation results. Providing stronger semantics would compromise resilience, extensibility, and observability.
- **No exclusive owners** — resources may be read and written by multiple parties and components, often responsible for orthogonal concerns. Controllers cannot assume their decisions will not be overridden and must continually verify assumptions.
- **Status is reconstructable by observation** — controllers reconcile observed and desired state and repair discrepancies. Kubernetes avoids maintaining opaque, internal persistent state.
- **Object references use predictable names** — references use client-provided, immutable names to facilitate loose coupling, declarative references, disaster recovery, and deletion and re-creation. References are represented as tuples of name, namespace, API version, and resource type rather than URLs.

API Topology

The API URL structure follows this form:

```
/prefix/group/version/namespaces/namespace/resourcetype/name
```

The API is divided into **groups** of related resource types that co-evolve together across API **versions**. Resource instances are grouped by user-created **namespaces**, which scope names, references, and some policies.

All resources contain common **metadata**, including the information in the URL path. Metadata also includes user-provided key-value pairs in the form of **labels** (for filtering and grouping by identifying attributes) and **annotations** (generally used by extensions for configuration and checkpointing).

Most resources contain a **spec** (desired state) and **status** (observed state). Status is written using the `/status` subresource, using the same API schema, to enable distinct authorization policies for users and controllers.

```
apiVersion: v1
kind: Pod
metadata:
  namespace: default
```

```
  name: explorer
  labels:
    category: demo
  annotations:
    commit: 483ac937f496b2f36a8ff34c3b3ba84f70ac5782
spec:
  containers:
    - name: explorer
      image: gcr.io/google_containers/explorer:1.1.3
      args: ["-port=8080"]
      ports:
        - containerPort: 8080
          protocol: TCP
status:
```

Each API server supports a discovery API to enable clients to discover available API groups, versions, and types, and also publishes OpenAPI schemas for documentation and validation.

Resource Lifecycle

Each API resource undergoes a common sequence of behaviors upon mutation:

1. **Authentication** — verifies the caller's identity
2. **Authorization** — applies identity-based policies (built-in RBAC or administrator-defined webhook)
3. **Defaulting** — API-version-specific default values are made explicit and persisted
4. **Conversion** — the apiserver converts between the client-requested API version and the storage version
5. **Admission control** — built-in and administrator-defined resource-type-specific policies
6. **Validation** — resource field values are validated
7. **Idempotence** — resources are accessed via immutable, client-provided names
8. **Optimistic concurrency** — writes may specify a precondition that the `resourceVersion` has not changed
9. **Audit logging** — records the sequence of changes by all actors

Additional behaviors are supported upon deletion:

- **Graceful termination** — some resources support delayed deletion, indicated by `deletionTimestamp` and `deletionGracePeriodSeconds`
- **Finalizers** — a block on deletion placed by an external controller, which must be removed before deletion can complete
- **Garbage collection** — a resource may specify `ownerReferences`; the resource is deleted once all referenced owners have been deleted

And for reads:

- **List** — all resources of a type within a namespace may be requested; response chunking is supported
- **Label selection** — lists may be filtered by label keys and values
- **Watch** — a client may subscribe to changes using the `resourceVersion` returned with list results

For how controllers interact with these APIs, see Week 1.2. For conditions and status conventions in depth, see Week 2.1.

References

- Kubernetes Resource Model (KRM)

API Request Lifecycle

Every interaction with a Kubernetes cluster — whether from `kubectl`, an internal controller, or an external operator — passes through the kube-apiserver's request pipeline. Understanding these stages is essential for building secure, efficient, and extension-ready Kubernetes applications.

The Request Pipeline

When a request enters the kube-apiserver, it does not immediately interact with etcd. Instead, it traverses a chain of stages, each responsible for a distinct concern. For a mutation, the stages execute in the following order:

1. **Authentication** — validates the caller's identity using X.509 client certificates, bearer tokens, or OIDC tokens. If authentication fails, the request is rejected with a 401 Unauthorized response. For a deeper treatment, see Week 3.2.
2. **Audit** — records the request for compliance and debugging purposes. Every request is logged, providing a full trail of who did what and when.
3. **Impersonation** — allows authenticated users to act as another user or group. The apiserver checks whether the caller has the authority to impersonate the target identity.
4. **Priority and Fairness** — classifies requests into flow schemas and priority levels to prevent API server overload. Requests are categorized (for example, leader election versus background listing) and rate-limited to ensure cluster stability.

Priority and Fairness is how the API server prevents a misbehaving controller from starving other clients. Without this mechanism, a single controller issuing a flood of list calls could degrade responsiveness for the entire cluster.

5. **Authorization** — checks whether the authenticated identity is allowed to perform the requested action on the specified resource. Kubernetes supports multiple authorization modes: RBAC, Node, Webhook, and ABAC.
6. **Mutation** — mutating admission webhooks can modify the request object before it is validated. This is where sidecars are injected, default values are set by policy engines, and labels are added automatically.

7. **Validation** — schema validation and validating admission webhooks ensure the object meets all constraints. Unlike mutating webhooks, validating webhooks cannot change the object — they can only accept or reject it.
8. **Storage** — the validated object is serialized and persisted to etcd. At this point the desired state is durably recorded and available for controllers to act on.

For a detailed treatment of admission webhooks, see Week 3.1. For authentication and authorization in depth, see Week 3.2.

A vulnerability in HTTP/2 (CVE-2023-44487, the "rapid reset" attack) led the client-go library to fall back to HTTP/1.1 connections by default. This is worth keeping in mind when debugging connection behavior between clients and the apiserver.

Exploring the API

The kube-apiserver exposes discovery endpoints that let you inspect which API groups, versions, and resource types are available. This is a useful way to verify that your request is reaching the apiserver and to understand the shape of the API surface.

Start a local proxy to the API server:

```
kubectl proxy --port=8001 &
```

List all API groups:

```
curl -s http://localhost:8001/apis | jq '.groups[].name'
```

Get the aggregated discovery document in v2 format:

```
curl -s -H "Accept: application/json;v=v2" http://localhost:8001/api | jq
```

Inspect a specific resource type (for example, pods in the default namespace):

```
curl -s http://localhost:8001/api/v1/namespaces/default/pods | jq
'.items[].metadata.name'
```

These commands confirm that your request successfully passes through the authentication and authorization stages described above. The discovery endpoints are also what client libraries like client-go use under the hood to negotiate API versions and resource schemas.

References

- Admission Controllers Reference
- The Life of a Kubernetes API Request

Declarative Configuration Layers

Declarative configuration management tools let you declare a desired state for some system and then automatically compare that desired state to reality, updating the managed system to match. The three-layer model provides a useful framework for understanding how these tools are structured — and why Kubernetes makes the architectural choices it does.

A Three-Layer Model

Any declarative configuration management system can be decomposed into three distinct layers:

Layer 1: Resource Schema

The resource schema defines which types of resources the tool can manage and what properties they have. For Puppet, this includes types like `file` and `mount`; for Terraform, it includes `aws_instance` and `google_compute_network`.

Kubernetes has perhaps the most explicit resource schema of all: it publishes the schema of all core resource types as a protocol buffer definition, and custom resource types may provide an OpenAPIv3 validation schema to document and enforce their shape. CRDs extend this schema dynamically without requiring a new API server.

Layer 2: Concrete Resource Syntax

The resource schema tells us what types of resources exist, but we need a concrete syntax to express specific instances — something we can type into an editor, check into version control, and pass to the tool.

In systems like Puppet and Terraform, this problem is solved with a custom language that supports both defining resources and providing abstraction (modules, variables, loops). This approach is convenient but often results in the slow, ad-hoc growth of "real language" features as the DSL accumulates complexity.

Kubernetes takes a radically different approach: the only concrete syntax the tool itself accepts is raw JSON or YAML defining fully concrete resources. The problem of abstraction — parameterization, iteration, composition — is pushed entirely to external tools. The open-source community has produced several such tools, including Helm, Kustomize, and Jsonnet.

Layer 3: Convergence Engine

On the far side from the concrete syntax is the convergence engine, whose job is to take a concrete resource catalog and update the managed system to match. Declarative configuration management derives both its power and many of its operational pitfalls from this layer: the configuration specifies only the desired state, and the engine must figure out how to get there.

Of all the systems in this space, Kubernetes has perhaps the most explicit notion of this layer as a distinct concern, under the name of **controllers**. Each controller watches a specific set of resource types and continuously reconciles observed state with desired state.

Why Strict Layering Matters

The strict separation between these layers — especially the boundary between concrete syntax and the resource schema — has several important implications:

Testability: If you can run configuration generation entirely separately from convergence, you can write tests that generate a catalog and make assertions about it without ever touching a live cluster. A CI pipeline can diff the generated manifests against a known-good baseline to catch unintended changes before they reach production.

Confidence in refactors: Pure refactors can be pursued with exceptional confidence. As long as a new version of your configuration tooling produces the same resource catalog, you can be certain the change had no functional effect on the cluster.

Separation of concerns: Kubernetes handles resource schemas (what types exist and what shape they have) and convergence (controllers that reconcile state). Users are free to choose whatever abstraction tool best fits their needs — a general-purpose programming language, a templating engine, or plain YAML for simple cases. The problem of "defining 10 pods with similar shapes but some parameters filled in" is a problem entirely like those encountered in other programming domains, and one adequately solved by functions and variables in any language.

Comparison with Other Systems

The three-layer model clarifies the differences between declarative configuration management tools:

System	Concrete Syntax	Convergence Engine
Puppet / Chef	Custom DSL with built-in abstraction primitives (modules, recipes). Conflates resource definition with abstraction, leading to ad-hoc language growth.	Built-in agent applies a catalog to a single host.
Terraform	HCL provides its own abstraction layer (modules, variables, for_each). Tightly coupled to the resource schema through providers.	Built-in plan/apply engine compares state files against reality.
Kubernetes	Raw JSON/YAML only. Abstraction is pushed to external tools (Helm, Kustomize, Jsonnet, or any programming language).	Controllers — decoupled, per-resource-type convergence engines that run as independent control loops.

Kubernetes achieves the purest separation of the three. By refusing to embed abstraction into its core, it keeps the API surface simple and lets developers write abstractions in tools and styles they already understand.

For the initial orientation on what Kubernetes really is, see Week 0. For how kubebuilder generates resource schemas (CRDs), see Week 2.1.

References

- Kubernetes Resource Management Design Proposal
- The Architecture of Declarative Configuration Management

Week 1.2 — Controllers, Informers, client-go

This module covers the fundamental building blocks that power every Kubernetes operator and controller. We begin with the architecture of **client-go**, the official Go client library, examining its core components — Reflector, DeltaFIFO, Indexer, and SharedInformerFactory — and how they form a pipeline from the API server to your controller logic. From there we explore the Informer/Lister/Workqueue pattern that decouples event delivery from reconciliation, walk through the official sample-controller line by line, and discuss production pitfalls that surface only at scale. These components form the foundation that controller-runtime (covered in Week 2.1) abstracts over. A solid understanding of this layer is essential for debugging operator issues and for knowing when the higher-level abstractions are helping versus hiding problems.

client-go Architecture

What is client-go?

client-go is the official Go client library for the Kubernetes API server. It provides the event-driven programming primitives that every controller and operator is built on — even `kubectl` uses client-go under the hood. If you are building a Kubernetes controller, this is the library you need to understand.

Conceptually, three layers sit between a developer and the API server:

1. **API server** — the source of truth for cluster state.
2. **client-go** (`k8s.io/client-go`) — provides typed clients, informers, listers, caches, and workqueues.
3. **controller-runtime** — a higher-level framework (covered in Week 2.1) that masks the details of client-go's event-oriented architecture.

The key packages within client-go are `tools/cache` (informers, reflectors, indexers, DeltaFIFO), `kubernetes` (typed clientsets), `dynamic` (untyped clients), and `discovery` (API group discovery).

Core Components

Every client-go-based controller relies on the following components, all defined under `tools/cache`.

Reflector

The **Reflector** (defined in `tools/cache/reflector.go`) mirrors API server state into a local store using `ListAndWatch`. On startup it performs an initial LIST to fetch all existing objects and records the `resourceVersion`. It then opens a long-running WATCH connection starting from that `resourceVersion` to receive incremental updates. When a new or modified resource is detected through the WATCH, the Reflector places the object — along with the type of change — into a DeltaFIFO queue inside its `watchHandler` function.

DeltaFIFO

DeltaFIFO is a producer-consumer queue that stores resource events as incremental deltas. Each entry records the change type (Added, Updated, Deleted, Replaced, Sync) along with the object itself. The Reflector is the producer; the Informer's `processLoop` is the consumer.

DeltaFIFO provides two guarantees that matter for controller correctness:

- **Exactly-once, in-order processing** per resource — deltas for a given key are consumed in the order they arrived.
- **Deduplication** — if multiple updates arrive for the same key before the consumer processes it, the queue coalesces them so the consumer sees the latest state.

Indexer

The **Indexer** (defined in `tools/cache/index.go`) is a thread-safe local cache that stores objects popped from DeltaFIFO. It uses the default key function `MetaNamespaceKeyFunc` (from `tools/cache/store.go`) to generate keys in the format `<namespace>/<name>`.

Multiple controllers within the same process share the Indexer cache. Mutating a cached object — even a shallow field like `Annotations` — without a deep copy corrupts the object for every controller that reads it. Always use `api.Scheme.Copy` or the generated `DeepCopy()` method before modifying a cached object.

Each resource stored in etcd has a hard size limit of approximately 1.5 MB. Keep this in mind when designing custom resources with large specs or status fields.

SharedInformerFactory

A **SharedInformerFactory** multiplexes watchers across controllers so that only one API server connection is maintained per resource type within a process. This saves API connections, serialization and deserialization costs, and server-side memory. Factory methods are available in `k8s.io/client-go/informers/factory.go`.

```
type SharedInformer interface {
    // AddEventHandler registers handler functions that are called
    // when objects are added, updated, or deleted.
    AddEventHandler(handler ResourceEventHandler) (ResourceEventHandlerRegistration,
error)

    // HasSynced returns true after the initial LIST has been
    // fully processed and the cache is populated.
    HasSynced() bool

    // LastSyncResourceVersion returns the resourceVersion observed
    // during the most recent sync with the API server.
    LastSyncResourceVersion() string

    // Run starts the informer and blocks until the stop channel is closed.
```

```
    Run(stopCh <-chan struct{})
}
```

The `SharedInformer` interface is the contract that all informers implement. Calling `AddEventHandler` registers your controller's callbacks; calling `Run` starts the Reflector and begins populating the cache.

The Data Flow Pipeline

The following diagram shows how data flows from the API server through client-go into your controller logic:

```
API Server
    |
    | (ListAndWatch)
    v
Reflector
    |
    | (Enqueue deltas)
    v
DeltaFIFO
    |
    | (Pop + store)
    v
Informer (Indexer / Cache)
    |
    | (Invoke registered callbacks)
    v
Event Handlers (AddFunc / UpdateFunc / DeleteFunc)
    |
    | (Enqueue namespace/name key)
    v
Workqueue
    |
    | (Dequeue key)
    v
Controller Sync Handler (your reconciliation logic)
```

The top half of this pipeline — Reflector, DeltaFIFO, Indexer — is managed entirely by client-go. The bottom half — event handlers, workqueue, and sync handler — is what you write as a controller author.

For a walkthrough of a controller that implements the bottom half, see Sample Controller Walkthrough. For how controller-runtime abstracts this entire pipeline behind `ctrl.NewControllerManagedBy()`, see Week 2.1.

References

- client-go tools/cache
- client-go under the hood
- Kubernetes Informers Presentation

Informers, Listers, and Workqueues

Your controller needs three things: **eyes** to watch for changes, **memory** to recall current state, and a **task list** to schedule work. In client-go, these roles map to Informers, Listers, and Workqueues.

Informers — The Eyes

An Informer watches the API server and maintains a local cache of the resources it tracks. Under the hood the Reflector performs two operations:

1. An initial **LIST** that fetches every existing object of the watched resource type and records the `resourceVersion`.
2. A continuous **WATCH** that streams incremental changes starting from that `resourceVersion`.

Every object returned by the initial LIST is treated as "new" and passed to the `AddFunc` handler. Subsequent creations go to `AddFunc`, modifications to `UpdateFunc`, and deletions to `DeleteFunc`.

The key implementation is **SharedIndexInformer** (defined in `tools/cache/shared_informer.go`), which ties together the Reflector, DeltaFIFO, the event processor, and the Indexer. This is what most people mean when they say "Informer."

Listers — The Memory

A Lister is a thin, read-only wrapper around the Indexer (the local cache). Instead of making a live API call to retrieve an object, you query the Lister:

```
pod, err := podLister.Pods(namespace).Get(name)
```

This read hits the in-process cache and returns immediately — no network round trip, no additional load on the API server, and the data is kept fresh by the Informer's WATCH stream.

In controller-runtime, `client.Get()` uses a cache-backed client by default, which is equivalent to using a Lister. Direct API calls should only be used when you need an uncached, strongly consistent read (for example, right before a write to avoid conflicts).

Workqueues — The Task List

When an event handler fires, it does **not** run the reconciliation logic inline. Instead, it extracts the object's key (`namespace/name`) and enqueues it into a workqueue. The reconciliation loop then

processes keys from the queue independently.

This design provides three critical benefits:

- **Deduplication** — if three rapid updates arrive for the same Pod, the queue collapses them into a single entry. The controller reconciles once against the current state, not three times against three stale snapshots.
- **Rate limiting** — failed items are requeued with exponential backoff, preventing a broken resource from consuming all worker capacity.
- **Serialized processing per key** — while multiple worker goroutines may run in parallel, the queue guarantees that only one worker processes a given key at a time.

Worker loop pseudocode:

```
while true:
    key = queue.Get()
    err = reconcile(key)
    if err == nil:
        queue.Forget(key)   // reset backoff
    else:
        queue.AddRateLimited(key)  // requeue with backoff
    queue.Done(key)  // release processing lock
```

Controller Guidelines

The upstream Kubernetes project maintains a set of guidelines for writing correct controllers. Here are the most important ones.

1. **Operate on one item at a time via workqueue.** Use `workqueue.Interface` for deduplication and safe concurrent processing. When a secondary resource (such as a Pod) triggers an event, find its owning resource (such as a ReplicaSet) and enqueue that instead.
2. **Random ordering between resources.** There is no ordering guarantee across distinct watches. If `resourceA/X` is created before `resourceB/Y`, your controller may observe them in reverse order.
3. **Level-driven, not edge-driven.** Your controller may be offline for an arbitrary period. You cannot count on seeing a transition from `false` to `true` — only the current value. Reconcile based on the current state of the world, not the event that triggered the reconciliation.
4. **Use SharedInformers.** Never create individual informers per controller. SharedInformers provide shared caches and multiplex API connections, saving server resources.
5. **Never mutate original objects from the cache.** Caches are shared across controllers. Always deep copy before modifying.
6. **Wait for secondary caches before processing items.** Call `cache.WaitForCacheSync` at startup. Processing items before caches are populated leads to thrashing (for example, a ReplicaSet controller that sees zero Pods because the Pod cache has not synced yet).
7. **Expect other actors.** Your controller is not the only entity modifying resources. The current state can change at any moment — guard against assumptions about what "should" be there.

8. **Run leader election for availability.** Deploy multiple controller replicas via a Deployment and use `k8s.io/client-go/tools/leader-election` so that exactly one is active at a time.

Resync

Informers can optionally re-list all cached objects on a periodic interval known as the **resync period**. The default resync period is typically around 10 hours.

During a resync, the Informer calls the `UpdateFunc` handler for every object in the cache, even if nothing has changed on the API server. This acts as a safety net — if a reconciliation was missed or an error was swallowed, the resync gives the controller another chance to process the object.

A resync does **not** make a new LIST call to the API server. It simply re-delivers the existing cached objects to the event handlers. If you need to avoid redundant reconciliations during resync, compare the `ResourceVersion` of the old and new objects in your `UpdateFunc` handler and skip processing if they are equal — but be careful, because skipping too aggressively can prevent retries of previously failed items.

References

- Informers, Listers & Workqueues

Sample Controller Walkthrough

The sample-controller is the official Kubernetes example operator. It shows you how to build a controller using client-go from scratch, managing a custom resource called **Foo** that creates and manages Deployments.

The Foo Custom Resource

The sample-controller defines a custom resource called `Foo`. Each `Foo` object declares a Deployment name and a desired replica count. The controller ensures that a corresponding Deployment exists with the correct number of replicas.

```
type Foo struct {
    metav1.TypeMeta   `json:",inline"`
    metav1.ObjectMeta `json:"metadata,omitempty"`

    Spec   FooSpec   `json:"spec"`
    Status FooStatus `json:"status"`
}

type FooSpec struct {
    DeploymentName string `json:"deploymentName"`
    Replicas       *int32 `json:"replicas"`
}
```

```
type FooStatus struct {
    AvailableReplicas int32 `json:"availableReplicas"`
}
```

After defining the types in `types.go`, the project uses `code-generator` (via `./hack/update-codegen.sh`) to produce typed clients, informers, listers, and `DeepCopy` methods automatically.

Program Entrypoint

The `main.go` file sets up all the components the controller needs:

```
kubeClient, err := kubernetes.NewForConfig(cfg)
exampleClient, err := clientset.NewForConfig(cfg)

kubeInformerFactory := kubeinformers.NewSharedInformerFactory(kubeClient,
time.Second*30)
exampleInformerFactory := informers.NewSharedInformerFactory(exampleClient,
time.Second*30)

controller := NewController(ctx, kubeClient, exampleClient,
    kubeInformerFactory.Apps().V1().Deployments(),
    exampleInformerFactory.Samplecontroller().V1alpha1().Foos())

kubeInformerFactory.Start(ctx.Done())
exampleInformerFactory.Start(ctx.Done())

controller.Run(ctx, 2)
```

There are two clients because the controller works with two API groups: the built-in `apps/v1` (for Deployments) and the custom `samplecontroller/v1alpha1` (for Foos). Each client gets its own `SharedInformerFactory`. The factories are started to begin populating caches, and then the controller runs with two worker goroutines.

Event Handler Registration

The `NewController` function registers event handlers on both the Foo and Deployment informers.

Foo informer handlers — every Foo event (add, update) enqueues the Foo itself:

```
fooInformer.Informer().AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc: controller.enqueueFoo,
    UpdateFunc: func(old, new interface{}) {
        controller.enqueueFoo(new)
    },
})
```

Deployment informer handler — when a Deployment changes, the controller checks whether it is

owned by a Foo and, if so, enqueues the owning Foo:

```
deploymentInformer.Informer().AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc: controller.handleObject,
    UpdateFunc: func(old, new interface{}) {
        newDepl := new.(*appsv1.Deployment)
        oldDepl := old.(*appsv1.Deployment)
        if newDepl.ResourceVersion == oldDepl.ResourceVersion {
            return
        }
        controller.handleObject(new)
    },
    DeleteFunc: controller.handleObject,
})
```

The `handleObject` function extracts the owner reference and, if the owner is a Foo, enqueues it:

```
func (c *Controller) handleObject(obj interface{}) {
    // ... tombstone handling omitted for brevity ...
    if ownerRef := metav1.GetControllerOf(object); ownerRef != nil {
        if ownerRef.Kind != "Foo" {
            return
        }
        foo, err := c.foosLister.Foos(object.GetNamespace()).Get(ownerRef.Name)
        if err != nil {
            return  // orphaned object; ignore
        }
        c.enqueueFoo(foo)
    }
}
```

There is no `DeleteFunc` on the Foo informer. When a Foo is deleted, Kubernetes garbage collection automatically deletes the owned Deployment (via OwnerReferences), which is sufficient to clean up.

The Work Loop

The controller's `Run` method waits for caches to sync and then launches worker goroutines. Each worker repeatedly calls `processNextWorkItem`:

```
func (c *Controller) processNextWorkItem(ctx context.Context) bool {
    objRef, shutdown := c.workqueue.Get()
    if shutdown {
        return false
    }
    defer c.workqueue.Done(objRef)
```

```
    err := c.syncHandler(ctx, objRef)
    if err == nil {
        c.workqueue.Forget(objRef)
        return true
    }

    utilruntime.HandleErrorWithContext(ctx, err,
        "Error syncing; requeuing for later retry",
        "objectReference", objRef)
    c.workqueue.AddRateLimited(objRef)
    return true
}
```

The three workqueue calls serve distinct purposes:

- `Get()` — dequeues the next key and marks it as being processed (no other worker will pick up the same key).
- `Done(key)` — releases the processing lock so the key can be dequeued again if it was requeued.
- `Forget(key)` — resets the rate-limiting backoff counter for this key (used on success).
- `AddRateLimited(key)` — requeues with exponential backoff (used on failure).

The Sync Handler

The `syncHandler` is where the actual reconciliation logic lives. It follows a clear sequence:

1. **Parse the key** into namespace and name.
2. **Get the Foo** from the lister (local cache, not an API call).
3. **Get or create the Deployment** — look up the Deployment by the name specified in `foo.Spec.DeploymentName`. If it does not exist, create it.
4. **Check OwnerReference** — verify the Deployment is controlled by this Foo. If another resource owns it, emit a warning event and return an error.
5. **Reconcile replicas** — if the desired replica count differs from the actual count, update the Deployment.
6. **Update status** — set `foo.Status.AvailableReplicas` to reflect the Deployment's actual state.

```
func (c *Controller) syncHandler(ctx context.Context, objectRef cache.ObjectName)
error {
    foo, err := c.foosLister.Foos(objectRef.Namespace).Get(objectRef.Name)
    if errors.IsNotFound(err) {
        return nil  // Foo was deleted; nothing to do
    }
    if err != nil {
        return err
    }

    deploymentName := foo.Spec.DeploymentName
```

```
    deployment, err :=
c.deploymentsLister.Deployments(foo.Namespace).Get(deploymentName)
    if errors.IsNotFound(err) {
        deployment, err = c.kubeclientset.AppsV1().Deployments(foo.Namespace).Create(
            ctx, newDeployment(foo), metav1.CreateOptions{FieldManager: FieldManager})
    }
    if err != nil {
        return err
    }

    if !metav1.IsControlledBy(deployment, foo) {
        msg := fmt.Sprintf(MessageResourceExists, deployment.Name)
        c.recorder.Event(foo, corev1.EventTypeWarning, ErrResourceExists, msg)
        return fmt.Errorf("%s", msg)
    }

    if foo.Spec.Replicas != nil && *foo.Spec.Replicas != *deployment.Spec.Replicas {
        deployment, err = c.kubeclientset.AppsV1().Deployments(foo.Namespace).Update(
            ctx, newDeployment(foo), metav1.UpdateOptions{FieldManager: FieldManager})
    }
    if err != nil {
        return err
    }

    err = c.updateFooStatus(foo, deployment)
    return err
}
```

This pattern — get from cache, compare desired vs. actual, create or update, then update status — is the standard reconciliation loop that every client-go controller follows.

For a hands-on exercise building a controller from scratch using this same pattern, see Exercise 1.

References

- How to Write a Kubernetes Operator Using client-go
- Writing Kube Controllers for Everyone - Maciej Szulik

Informer Pitfalls and Error Handling

Informers are elegant in small clusters but introduce subtle failure modes at scale. This page covers the most common pitfalls, drawn from the upstream Kubernetes controller guidelines and from production experience running large clusters.

Level-Driven, Not Edge-Driven

This is the single most important principle for writing correct controllers.

A **level-driven** controller examines the current state of the world and determines what action to

take. An **edge-driven** controller examines the transition (the delta between old and new state) and reacts to what changed.

Do **not** compare `oldObj` and `newObj` in your `UpdateFunc` handler to decide whether to reconcile. The `oldObj` parameter in `UpdateFunc` is a footgun — it tempts you into edge-driven logic that breaks when events are missed, reordered, or coalesced.

Edge-driven logic fails because:

- Informers guarantee that you will eventually see the current state, but they do **not** guarantee that you will see every intermediate state.
- During high churn, multiple updates may be coalesced into a single event.
- If your controller restarts, it receives the full current state via the initial LIST — there is no "previous state" to compare against.

The correct approach: your `UpdateFunc` handler should enqueue the object's key unconditionally. The sync handler should then read the current state from the cache (or API server if needed) and reconcile from scratch.

Informer Pitfalls at Scale

The following failure modes emerge in production clusters with tens of thousands of resources.

The Initial LIST Phase

When an Informer starts, it performs a LIST that returns **all** existing objects of the watched type. In a cluster with 100,000 Pods, the Informer loads 100,000 objects into memory at once, and each one triggers an `AddFunc` call.

Serial Handler Execution

Event handlers run **serially** on an unbounded ring buffer. If an `AddFunc` handler performs a slow operation (a network call, a database write), events queue up in memory while waiting their turn.

The OOM Loop

These two properties combine into a pathological failure mode observed in production:

1. The initial LIST returns a large number of objects, all enqueued in the ring buffer.
2. The handler is slow on certain items (for example, it makes a blocking API call per object).
3. While the handler works through the backlog, the WATCH stream continues generating new events that accumulate in the buffer.
4. Periodic resync compounds the problem by re-delivering every cached object.
5. Memory grows until the process is OOM-killed.
6. Kubernetes restarts the Pod, and the cycle repeats.

Mitigations

- **Use a workqueue.** The first recommendation in the official controller guidelines. Move processing out of the event handler and into a rate-limited workqueue with metrics, retries, and backoff.
- **Filter with label selectors.** Use `WithTweakListOptions` when creating a `SharedInformerFactory` to restrict the LIST and WATCH to only the objects your controller cares about.
- **Reduce object size.** Use the `SetTransform` hook on the Informer to strip out fields your controller does not need (such as managed fields or large annotations). This reduces memory footprint but can complicate debugging.

Error Handling and Requeues

Correct error handling follows directly from the upstream guidelines.

Percolate errors to the top

Do not silently swallow errors inside your sync handler. Return the error so that the work loop can decide what to do with it. A returned error triggers `workqueue.AddRateLimited`, which requeues the item with exponential backoff.

```
func (c *Controller) processNextWorkItem(ctx context.Context) bool {
    objRef, shutdown := c.workqueue.Get()
    if shutdown {
        return false
    }
    defer c.workqueue.Done(objRef)

    err := c.syncHandler(ctx, objRef)
    if err == nil {
        c.workqueue.Forget(objRef)  // success: reset backoff
        return true
    }

    // failure: log and requeue with backoff
    utilruntime.HandleErrorWithContext(ctx, err,
        "Error syncing; requeuing for later retry",
        "objectReference", objRef)
    c.workqueue.AddRateLimited(objRef)
    return true
}
```

The three workqueue calls

Call	Purpose
`Done(key)`	Releases the processing lock. Must always be called (typically via `defer`) so the key can be picked up again if it was requeued.

Call	Purpose
`Forget(key)`	Resets the rate-limiting backoff counter for this key. Call this on **success** so the next event for this key is processed immediately.
`AddRateLimited(key)`	Requeues the key with exponential backoff. Call this on **failure** so the controller retries after an increasing delay.

Common Mistakes

The following mistakes come up frequently in production controllers and during code reviews.

Mutating cached objects without deep copy

The Informer cache is shared across all controllers in the same process. If you modify an object retrieved from the cache — even a map field like `Annotations` on a shallow copy — every other controller reading that object sees the corrupted data. Always call `DeepCopy()` before modifying.

Not waiting for cache sync

Starting to process items before `cache.WaitForCacheSync` returns leads to incorrect decisions. For example, a ReplicaSet controller that begins reconciling before the Pod cache is populated will see zero Pods and create duplicates.

Making direct API calls instead of using the Lister

The entire point of the Informer cache is to avoid hitting the API server on every reconciliation. If your sync handler calls `clientset.CoreV1().Pods(ns).Get(…)` instead of `podLister.Pods(ns).Get(name)`, you defeat the cache and add unnecessary load to the API server.

Use the Lister for reads and the clientset for writes. The Lister returns data from the local cache (fast, no network). The clientset talks to the API server (required for creates, updates, and deletes).

Using the cache as a source of truth for writes

The Informer cache is eventually consistent. Before performing a write (update or patch), re-GET the resource from the API server to get the latest `resourceVersion`. Writing with a stale `resourceVersion` results in a conflict error. While the workqueue retry mechanism will eventually resolve this, it is more efficient to re-GET first.

References

- Kubernetes Informers Are So Easy… to Misuse!
- Learning Concurrent Reconciling
- Writing Controllers

Week 2.1 — Controllers with controller-runtime

This week moves from the raw client-go informer and workqueue machinery covered in Week 1.2 to the higher-level abstractions that most production controllers use. We begin with the foundational terminology — Groups, Versions, Kinds, and Resources — that underpins every Kubernetes API interaction. From there we explore controller-runtime and Kubebuilder, which replace boilerplate informer setup with a single `Reconcile` method. We then cover API design conventions: the spec/status split, condition reporting, and the `observedGeneration` pattern that prevents stale status. Validation and linting ensure your CRD schemas follow upstream Kubernetes conventions before they reach review. Finally, we compare Update, the three Patch types, and Server-Side Apply, explaining when each is appropriate for controllers and CI/CD systems alike.

GVK, GVR, and API Terminology

When working with the Kubernetes API, four terms appear everywhere: **Groups**, **Versions**, **Kinds**, and **Resources**. Understanding how they relate — and the shorthand GVK and GVR — is essential before writing controllers.

Groups

An API Group is a logical collection of related functionality. Core resources live in the empty group (often written as `core` or `""`), while others belong to named groups such as `apps`, `batch`, or `networking.k8s.io`. Group names follow DNS subdomain conventions; the Kubernetes project reserves single-word names and the `*.k8s.io` suffix.

Versions

Each group exposes one or more versions (e.g. `v1`, `v1beta1`). Versions indicate the stability and maturity of the API surface. A Kind may change shape between versions, but every version must be able to round-trip all data stored by other versions so that upgrading or downgrading never loses information.

Kinds

A **Kind** is the type name of an API object — always CamelCase and singular (e.g. `Deployment`, `Pod`, `CronJob`). The API server uses the `kind` and `apiVersion` fields in every JSON payload to determine which schema to apply.

Resources

A **Resource** is the lowercase, plural form that appears in REST URLs (e.g. `deployments`, `pods`, `cronjobs`). In most cases there is a one-to-one mapping between Kind and Resource, but exceptions exist. For example, the `Scale` Kind is returned by multiple resources such as `deployments/scale` and `replicasets/scale`, which is what allows the HorizontalPodAutoscaler to work across different resource types.

GVK vs GVR

GroupVersionKind (GVK) identifies the Go type — for example `apps/v1.Deployment`. **GroupVersionResource (GVR)** identifies the REST endpoint — for example `apps/v1/deployments`.

The **REST mapper** translates between GVK and GVR. When you call `client.Get` in controller-runtime, the framework uses the GVK (looked up via the Scheme) to find the correct GVR and issue the HTTP request.

The Scheme

The `Scheme` is a registry that maps Go types to their GVKs for serialization and deserialization. When you register a type — for instance `"batch.tutorial.kubebuilder.io/v1".CronJob{}` — the Scheme records that this Go struct corresponds to the `CronJob` Kind in the `batch.tutorial.kubebuilder.io/v1` group-version. This allows the framework to construct a `&CronJob{}` from JSON or to determine the correct API path when submitting an update.

```
import "k8s.io/apimachinery/pkg/runtime"

scheme := runtime.NewScheme()
batchv1.AddToScheme(scheme) // registers CronJob GVK
```

GVK uses CamelCase singular (`CronJob`), GVR uses lowercase plural (`cronjobs`). When using controller-runtime, `client.Get` resolves the GVK through the Scheme internally — you rarely need to work with GVR directly.

Summary

Concept	Example	Purpose
Group	`batch`, `apps`, `networking.k8s.io`	Logical collection of related API types
Version	`v1`, `v1beta1`	Stability level; enables API evolution
Kind (GVK)	`batch/v1.CronJob`	Identifies the Go type and schema
Resource (GVR)	`batch/v1/cronjobs`	Identifies the REST endpoint
Scheme	`runtime.Scheme`	Maps Go types to GVKs for (de)serialization

References

- Groups and Versions and Kinds, oh my!
- API Conventions

Controller-runtime and Kubebuilder

From client-go to controller-runtime

In Week 1.2 we built controllers by manually wiring up informers, listers, and work queues. Controller-runtime abstracts all of that machinery behind a single interface: you implement a `Reconcile(ctx, req) (Result, error)` method, and the framework handles event filtering, caching, leader election, and worker management.

This dramatically reduces boilerplate while preserving the same level-triggered, eventually-consistent model underneath.

The Reconciler Pattern

Every controller-runtime controller is built around a **Reconciler** struct that embeds a client and a scheme:

```
type CronJobReconciler struct {
    client.Client
    Scheme *runtime.Scheme
}

func (r *CronJobReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    log := logf.FromContext(ctx)

    var cronJob batchv1.CronJob
    if err := r.Get(ctx, req.NamespacedName, &cronJob); err != nil {
        // If not found, the object was deleted -- nothing to do
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }

    // Reconciliation logic here
    return ctrl.Result{}, nil
}
```

The `Request` carries only a `NamespacedName` — not the object itself. You fetch the current state from the cache via `r.Get`, which prevents your controller from acting on stale data passed through event handlers.

RBAC Markers

Controllers running in-cluster need RBAC permissions. Kubebuilder generates ClusterRole manifests from marker comments placed above the `Reconcile` method:

```
//+kubebuilder:rbac:groups=batch.tutorial.kubebuilder.io,resources=cronjobs,verbs=get;
list;watch;create;update;patch;delete
//+kubebuilder:rbac:groups=batch.tutorial.kubebuilder.io,resources=cronjobs/status,ver
bs=get;update;patch
//+kubebuilder:rbac:groups=batch,resources=jobs,verbs=get;list;watch;create;update;pat
```

```
ch;delete
```

Running `make manifests` invokes controller-gen to produce `config/rbac/role.yaml` from these markers.

Builder Pattern — For, Owns, Watches

The `SetupWithManager` method declares what resources the controller cares about:

```
func (r *CronJobReconciler) SetupWithManager(mgr ctrl.Manager) error {
    return ctrl.NewControllerManagedBy(mgr).
        For(&batchv1.CronJob{}).
        Owns(&kbatch.Job{}).
        Complete(r)
}
```

- `For(&batchv1.CronJob{})` — the primary resource this controller reconciles. Every create, update, or delete of a CronJob enqueues a reconciliation request.
- `Owns(&kbatch.Job{})` — owned child resources. Changes to a Job trigger reconciliation of its **parent** CronJob. The framework automatically sets an OwnerReference on owned resources, which enables garbage collection when the parent is deleted.
- `Watches(…)` — arbitrary resources that are not owned by the primary. Useful when a controller needs to react to changes in ConfigMaps, Secrets, or other shared resources.

Controller-runtime auto-creates informers for any resource type used in `client.Get` or `client.List`, even if not declared in `For`/`Owns`/`Watches`. This can cause performance issues because cache warming blocks the worker goroutine. Set `ReaderFailOnMissingInformer: true` on the manager to catch accidental implicit watches.

Kubebuilder vs Operator SDK

Operator SDK uses Kubebuilder under the hood for Go-based operators. The `operator-sdk` CLI produces the same project scaffolding and uses the same controller-runtime library, so the generated code is nearly identical.

Operator SDK adds several features on top of the Kubebuilder baseline:

- **OLM integration** — packaging and lifecycle management via the Operator Lifecycle Manager
- **OperatorHub** — publishing operators to the community hub
- **Scorecard** — a testing tool that validates operator best practices
- **Ansible and Helm operators** — Operator SDK also supports building operators without writing Go, using Ansible playbooks or Helm charts

For pure Go operators, the choice between `kubebuilder init` and `operator-sdk init` is largely a

matter of which ecosystem tools you need. The underlying reconciler code, RBAC markers, and CRD generation are the same.

References

- What's in a Controller?
- Implementing a Controller
- Operator SDK FAQ

Spec, Status, and Conditions

Spec vs Status Convention

By convention, Kubernetes API objects separate desired state from observed state using two top-level fields:

- `spec` describes the desired state. It is written by users (and sometimes by controllers such as autoscalers). The spec is a complete description of intent, including user-provided values, system-expanded defaults, and properties set by other ecosystem components.
- `status` describes the observed state. It is written by the controller that manages the resource and should reflect the most recent observations of actual cluster state.

These fields are updated through separate subresources (`/status`), which enables distinct RBAC policies. Users can be granted full write access to `spec` and read-only access to `status`, while controllers get read-only access to `spec` but full write access to `status`.

The PUT and POST verbs on the main resource ignore `status` values to prevent accidental overwrites in read-modify-write scenarios. To update status, controllers use the `/status` subresource endpoint (via `r.Status().Update()` in controller-runtime).

The system is **level-based**, not edge-based. If `spec.replicas` changes from 2 to 5 and then to 3, the controller is not required to reach 5 before converging on 3.

Conditions

Conditions provide a standard mechanism for controllers to report higher-level status information. They allow tools and other controllers to collect summary data about a resource without understanding its internal details.

The upstream `metav1.Condition` struct defines the standard schema:

```
type Condition struct {
    // Type in CamelCase (e.g. "Available", "Progressing", "Degraded")
    Type               string
    // Status is one of True, False, Unknown
    Status             ConditionStatus
    // The .metadata.generation this condition was based on
```

```
    ObservedGeneration int64
    // When the condition last transitioned between statuses
    LastTransitionTime Time
    // Machine-readable CamelCase reason for the transition
    Reason             string
    // Human-readable message with transition details
    Message            string
}
```

Condition Conventions

- **Apply conditions early** — controllers should set their conditions on first visit, even with status `Unknown`, so that other components know the controller is aware of the resource and is making progress.
- **Positive polarity preferred** — condition types like `Ready` and `Available` (where `True` means healthy) are generally easier to reason about than negative-polarity types, though exceptions exist (e.g. `MemoryExhausted`).
- **Use PascalCase names** — condition types should be short PascalCase identifiers (e.g. `Ready` rather than `MyResourceReady`).
- **Top-level summary condition** — it is helpful to have a common top-level condition (often `Ready` for long-running resources or `Succeeded` for batch resources) that summarizes more detailed conditions.
- **Conditions over phase** — the older `phase` field pattern is deprecated. New APIs should use conditions instead, because adding new enum values to a phase field breaks backward compatibility.

Declaring Conditions in Your CRD

Conditions should be declared as a list with merge semantics on the `type` key:

```
// +listType=map
// +listMapKey=type
// +patchStrategy=merge
// +patchMergeKey=type
// +optional
Conditions []metav1.Condition `json:"conditions,omitempty"`
```

Controller Pitfalls for Status

Several common mistakes arise when controllers report status:

- **Always report conditions** — silence from a controller is ambiguous. If a controller does not set any conditions, consumers cannot tell whether the controller has not yet reconciled the resource or whether everything is healthy. Initialize all conditions to `Unknown` at the start of reconciliation.
- **Use observedGeneration** — set `ObservedGeneration` on every condition to the resource's current

`metadata.generation`. This lets consumers distinguish stale status from current status. A `Ready=True` condition means nothing on its own; it is meaningful only when `condition.observedGeneration == metadata.generation`.

- **Re-fetch after status updates** — Kubernetes uses optimistic concurrency via `resourceVersion`. After a status update changes the resource version, subsequent writes against the old version will fail with a conflict. Re-fetch the resource with `r.Get()` after calling `r.Status().Update()` to keep your reconciliation in sync.

```
if err := r.Status().Update(ctx, &cronJob); err != nil {
    return ctrl.Result{}, err
}
// Re-fetch to get the updated resourceVersion
if err := r.Get(ctx, req.NamespacedName, &cronJob); err != nil {
    return ctrl.Result{}, err
}
```

- **Reconstruct status from the world** — status should be derivable from the current state of child resources, not read from the previous status of the root object. This ensures correctness even after controller restarts.

References

- API Conventions

API Validation and Linting

Well-designed CRD schemas catch invalid input before it reaches your controller. Kubernetes offers multiple layers of validation, from declarative schema constraints to programmatic admission webhooks.

Validation Approaches

OpenAPI v3 Schema Validation

Every CRD includes an OpenAPI v3 schema that the API server enforces automatically. Kubebuilder generates this schema from your Go struct definitions and marker comments. Fields without proper validation markers accept any value that matches their Go type, which is rarely what you want.

CEL Validation Rules

Common Expression Language (CEL) rules let you express cross-field validation directly in the CRD schema. Kubebuilder supports these through the `+kubebuilder:validation:XValidation` marker:

```
// +kubebuilder:validation:XValidation:rule="self.minReplicas <=
self.maxReplicas",message="minReplicas must not exceed maxReplicas"
type AutoscalerSpec struct {
```

```
    MinReplicas int32 `json:"minReplicas"`
    MaxReplicas int32 `json:"maxReplicas"`
}
```

CEL rules run on the API server and do not require a webhook deployment.

Admission Webhooks

For validation logic that cannot be expressed declaratively — such as checking external state or enforcing complex business rules — you can implement validating admission webhooks. These are covered in more detail in Week 3.1.

Kube API Linter (KAL)

The Kube API Linter (KAL) is a Go-based static analysis tool that checks your CRD struct definitions against upstream Kubernetes API conventions. It catches mechanical API review issues so that human reviewers can focus on design.

Installation Options

KAL ships in three forms:

1. **Standalone binary** — build with `go build` and run directly
2. **golangci-lint module** — integrates into an existing golangci-lint v2 workflow via a `.custom-gcl.yml` configuration
3. **golangci-lint plugin** — loaded as a shared object by golangci-lint

Key Linters

KAL includes linters for the most common API convention violations:

Linter	What It Checks
`requiredfields`	Required fields are properly annotated
`statusoptional`	All status fields are marked `+optional`
`statussubresource`	Types with status fields enable the `/status` subresource
`conditions`	Condition fields follow the standard list/map conventions

Auto-Fix Support

Where fixes are straightforward, KAL can apply them automatically:

```
golangci-lint-kube-api-linter run path/to/api/types --fix
```

Common Validation Markers

Kubebuilder provides a rich set of markers for field-level validation. These translate directly into the OpenAPI schema in your CRD:

```
// +kubebuilder:validation:Minimum=0
// +kubebuilder:validation:Maximum=100
Replicas int32 `json:"replicas"`

// +kubebuilder:validation:MaxLength=256
// +kubebuilder:validation:MinLength=1
Name string `json:"name"`

// +kubebuilder:validation:Enum=Allow;Forbid;Replace
ConcurrencyPolicy string `json:"concurrencyPolicy"`

// +optional
Suspend *bool `json:"suspend,omitempty"`
```

Validation Guidelines from API Conventions

The upstream API conventions document provides detailed guidance:

- **String fields** — almost all should be checked for format and maximum length. Never perform case-insensitive comparisons in validation.
- **Numeric fields** — bound-check both minimum and maximum values.
- **List fields** — set maximum size and declare `listType` tags. Lists with set or map semantics should enforce uniqueness.
- **Map fields** — check maximum size and validate both keys and values.

Run KAL as part of your CI pipeline alongside `go vet` and `golangci-lint`. Catching convention violations early avoids costly API redesigns after your CRD is in production.

References

- API Conventions
- Kube API Linter (KAL)

Update, Patch, and Server-Side Apply

Kubernetes provides several mechanisms for modifying API objects. Choosing the right one depends on whether you are a user, a controller, or a CI/CD system, and whether multiple actors write to the same object.

Update (HTTP PUT)

An Update replaces the entire object. The client must GET the current object, modify it, and PUT the full object back. Kubernetes uses **optimistic concurrency**: the `resourceVersion` field is checked on every PUT, and the server rejects the write with a `409 Conflict` if the resource was modified since the GET.

This is the simplest approach for single-owner resources. The downside is that in a system with N concurrent actors, up to N-1 PUTs may fail and need retries, leading to O(N^2) requests in the worst case.

Patch Types

Patches send only the changes rather than the full object. Kubernetes supports three patch formats:

Type	Behavior	Use When
JSON Patch (RFC 6902)	A sequence of discrete operations: `add`, `remove`, `replace`, `move`, `copy`, `test`. Each operation targets a specific JSON pointer path.	You need precise, surgical changes to specific paths.
JSON Merge Patch (RFC 7386)	Merges a partial object into the existing one. Setting a field to `null` removes it. Lists are **replaced** entirely — there is no list merging.	Simple field updates where you do not need to merge list elements.
Strategic Merge Patch	A Kubernetes-specific format that uses `patchStrategy` and `patchMergeKey` struct tags to merge lists intelligently. For example, the `containers` list in a PodSpec merges on the `name` field. Lists without merge tags are replaced, just like JSON Merge Patch.	Default for client-side `kubectl apply`. Preferred when you need list merging on built-in types.

Strategic Merge Patch is **not supported** for custom resources. CRDs use JSON Merge Patch or Server-Side Apply instead.

Server-Side Apply

Server-Side Apply (SSA) moves the merge algorithm from the client to the API server and introduces **field ownership** tracking.

How It Works

- Each actor (called a **field manager**) sends a fully specified intent — the fields it cares about — via an Apply request.
- The API server records which field manager owns each field in `metadata.managedFields`.
- When two managers try to set the same field to different values, the server returns a **conflict**

instead of silently overwriting.

- No-op applies (where the object already matches the intent) do not bump `resourceVersion` or trigger watch events.

Two Controller Patterns

Controllers that adopt SSA typically follow one of two patterns:

Reconstructive

Build the desired state from scratch on every reconcile and Apply it. The server computes the diff. This is the recommended approach for new controllers — it is less error-prone because you always declare your full intent.

```
applyConfig := batchv1ac.CronJob(name, namespace).
    WithSpec(batchv1ac.CronJobSpec().
        WithSchedule("*/5 * * * *").
        WithJobTemplate(...))

err := r.Patch(ctx, applyConfig, client.Apply, client.FieldOwner("my-controller"),
client.ForceOwnership)
```

Extract-Modify-Apply

For existing controllers migrating from GET-modify-PUT, client-go provides an **extract** workflow: retrieve the current apply configuration for your field manager, modify it, and re-apply. This avoids accidentally claiming ownership of every field in the object (which the server will reject).

Force Conflicts

- **Controllers** should typically use `Force: true` (force conflicts) because a controller is the sole authority for the fields it manages and does not have a convenient way to surface conflict errors.
- **CI/CD systems** should **not** force conflicts. They need error paths to surface conflicts to humans — for example, when a hotfix applied by an SRE would be overwritten by an automated deployment.

When to Use SSA vs Update

Scenario	Recommendation
Single-owner resource (one controller manages all fields)	Traditional Update or full-object Patch is simpler
Multi-controller resource (multiple actors write different sections)	SSA with distinct field managers to prevent conflicts
CI/CD deploying manifests	SSA with conflict detection to catch drift and hotfixes

Scenario	Recommendation
Migrating from client-side `kubectl apply`	Server-side `kubectl apply --server-side` as a drop-in replacement

SSA is recommended for multi-controller resources where multiple actors write to the same object (e.g. one controller managing replicas while another manages annotations). For resources with a single owner, traditional Update is simpler and avoids the complexity of managed fields.

Summary

- **Update (PUT)** — full replacement with optimistic concurrency via `resourceVersion`.
- **JSON Patch** — surgical operations on specific paths.
- **JSON Merge Patch** — partial object merge; lists are replaced.
- **Strategic Merge Patch** — Kubernetes-aware list merging (built-in types only).
- **Server-Side Apply** — server-side merge with field ownership tracking and conflict detection; the recommended approach for multi-actor scenarios.

References

- Update API Objects Using kubectl patch
- Server-Side Apply in Kubernetes

Week 2.2 — Going Distributed

When multiple controller replicas manage shared resources, you step squarely into the territory of distributed systems. Race conditions, stale caches, and split-brain scenarios are no longer theoretical — they are daily operational realities. This week examines four layers of defense against these challenges. First, you will learn to structure your Reconcile loop so that every invocation is idempotent and tolerant of stale data. Next, you will study the theory behind distributed locks and understand why naive locking is insufficient for correctness. From there, you will implement Kubernetes-native leader election using Lease objects, and finally explore controller sharding as a strategy for scaling reconciliation beyond a single active replica.

Idempotent Reconciliation

A controller that cannot be safely re-run on the same input is a controller waiting to break. This section covers the structure, return semantics, and cache-awareness patterns that make reconciliation robust.

The Shape of Reconcile()

A well-structured `Reconcile` method follows a predictable flow that separates concerns and makes the logic easy to reason about:

1. **Fetch the resource** from the informer cache. If it is not found, it has been deleted — return early.
2. **Check for deletion**: if `metadata.deletionTimestamp` is set, run finalizer logic (clean up external resources), remove the finalizer, and return.
3. **Initialize the resource**: add a finalizer if missing, set default values, initialize status conditions.
4. **Perform core reconciliation**: create, update, or delete owned child resources to match the desired state declared in `spec`.
5. **Update status**: patch `status` to reflect the outcome. Compare against a `DeepCopy` taken before reconciliation to avoid unnecessary writes.

```
func (r *FooController) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result,
error) {
    var obj v1alpha1.Foo
    if err := r.Get(ctx, req.NamespacedName, &obj); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }
    if !obj.DeletionTimestamp.IsZero() {
        return r.finalize(ctx, &obj)
    }
    if err := r.ensureFinalizer(ctx, &obj); err != nil {
        return ctrl.Result{}, err
    }
    orig := obj.DeepCopy()
    reconcileErr := r.reconcileKind(ctx, &obj)
```

```
    if err := r.patchStatus(ctx, orig, &obj); err != nil {
        return ctrl.Result{}, err
    }
    return ctrl.Result{}, reconcileErr
}
```

Design for Idempotency

Every call to `Reconcile` should produce the same outcome for the same input state, regardless of how many times it runs or what event triggered it.

- **Do not depend on the triggering event.** The reconciler receives only a name and namespace — not the event type. Treat every invocation as "make reality match the spec."
- **Avoid unnecessary API calls.** Compare `status.observedGeneration` against `metadata.generation`. If they match and no external drift is possible, skip reconciliation entirely.
- **Make external calls conditional.** If your controller provisions cloud resources, check whether the resource already exists in the desired state before calling the provider API. A reconcile of an up-to-date object should be fast and make zero external calls.

Reconcile Return Values

The return value of `Reconcile` controls requeueing behavior:

Return Value	Meaning
`return ctrl.Result{}, nil`	Success. The object is not requeued unless a new event arrives.
`return ctrl.Result{}, err`	Error. The object is requeued with rate-limited exponential backoff.
`return ctrl.Result{Requeue: true}, nil`	Explicit requeue without error. Use sparingly — typically when an operation is in progress but has not failed.
`return ctrl.Result{RequeueAfter: time.Minute}, nil`	Periodic reconciliation on a wall-clock interval. Useful for polling external state that does not generate Kubernetes events.

Returning an error is the standard way to request a retry. Do not use `Requeue: true` as a substitute for proper error handling.

The Expectations Pattern

Because controller-runtime serves reads from a local informer cache, there is a window after a successful write where the cache has not yet observed the change.

Problem: You create a Pod, but the informer cache has not seen it yet. The next reconcile reads stale state, sees zero Pods, and creates a duplicate. This class of bug affects any controller that creates or deletes child resources.

Solution: The *expectations pattern* tracks expected writes in memory. After creating a Pod, the controller records an "expected create." On the next reconcile, it checks whether expectations are satisfied (the cache has caught up) before acting.

```
// Before creating children:
if !r.expectations.SatisfiedExpectations(key) {
    return ctrl.Result{}, nil // wait for cache to catch up
}
// After creating a child:
r.expectations.ExpectCreations(key, 1)
```

This pattern is used by the core ReplicaSet and Job controllers, and by the Elastic Cloud on Kubernetes operator. It is essential for any controller that manages a dynamic number of child resources.

References

- Writing Controllers
- Advanced Server-Side Apply: Patch vs Apply Controllers
- Common Pitfalls in Writing Kubernetes Controllers

Distributed Locking Theory

Distributed locks appear simple but conceal subtle failure modes. This section explains why locks are harder than they look and what guarantees are actually achievable.

Why Distributed Locks?

There are two distinct reasons to use a distributed lock, and they demand very different levels of rigor:

- **Efficiency:** Prevent duplicate expensive work. For example, two cron jobs should not both run the same batch computation simultaneously. If the lock mechanism fails, the worst case is wasted compute — the same work is done twice.
- **Correctness:** Prevent concurrent operations from corrupting shared state. For example, two processes must not simultaneously modify a file in object storage. If the lock mechanism fails, the worst case is data loss or inconsistency.

For efficiency-only use cases, a simple single-node lock (such as a Redis `SET … NX`) is sufficient. Correctness requires much stronger guarantees.

Why Locks Are Harder Than They Look

Three categories of failure undermine naive distributed locking:

- **Process pauses:** A lock holder may be paused by garbage collection, an OS context switch, or a page fault. During the pause, the lock expires and another process acquires it — but the original

process does not know it has lost the lock when it resumes.

- **Network delays:** Messages between a client and the lock service can be delayed arbitrarily. A "lock acquired" response may arrive after the lock has already expired.
- **Clock skew:** Nodes in a distributed system do not share a perfectly synchronized clock. Time-based lock expiration (TTLs) becomes unreliable when clocks drift or experience NTP jumps.

These failures are not edge cases. In production environments with garbage-collected runtimes (Java, Go), they occur regularly.

Fencing Tokens

The recommended solution for correctness-critical locking is the **fencing token** pattern:

1. The lock service issues a monotonically increasing *fencing token* each time a lock is acquired.
2. The client includes this token with every write to the shared resource.
3. The storage service rejects any write carrying a token lower than the highest token it has already seen.

This ensures that even if a stale lock holder wakes up after a pause, its writes are rejected because its token is outdated. The storage service acts as the final arbiter of ordering.

```
Client 1 acquires lock -> token 34
Client 1 pauses (GC)
Client 2 acquires lock -> token 35
Client 2 writes with token 35 -> accepted
Client 1 resumes, writes with token 34 -> rejected (34 < 35)
```

Kubernetes Leases

The `Lease` resource in the `coordination.k8s.io` API group provides a lightweight coordination primitive built into Kubernetes. It serves three built-in use cases:

1. **Node heartbeats:** Each node renews a Lease in the `kube-node-lease` namespace to signal liveness. The control plane uses `spec.renewTime` to detect unresponsive nodes.
2. **Leader election:** Controllers use Leases to elect a single active instance among multiple replicas. The holder sets `holderIdentity` and periodically renews the lease.
3. **API server identity:** Each kube-apiserver instance maintains a Lease in `kube-system`, enabling distributed features to discover and track active API servers.

Custom workloads can also create their own Lease objects for coordination. The best practice is to name Leases in a way that is clearly linked to the product or component (for example, `my-controller-leader`).

You can inspect the Leases in a cluster to observe these mechanisms in action:

```
# View node heartbeat leases
```

```
kubectl get leases -n kube-node-lease

# View control-plane leader election leases
kubectl get leases -n kube-system
```

Kubernetes leader election via Leases does **not** provide fencing token guarantees. Brief periods of dual leadership are possible — a paused leader can resume alongside a newly elected one. For most controllers, this is acceptable because reconciliation is designed to be idempotent (see Idempotent Reconciliation). The Kubernetes API server's conflict detection on stale `resourceVersion` values provides an additional layer of safety against state regression.

References

- How to Do Distributed Locking
- Leases

Leader Election in Kubernetes

Leader election allows multiple controller replicas to coordinate so that only one replica — the leader — actively reconciles resources at any given time. Standby replicas wait and take over if the leader becomes unresponsive.

How It Works

Leader election in Kubernetes uses a Lease object as the coordination mechanism:

1. A `Lease` object is created in a specific namespace (often the controller's own namespace).
2. Multiple controller replicas attempt to acquire the lease by setting `holderIdentity` to their own identity.
3. The Kubernetes API server's optimistic concurrency control ensures that only one replica succeeds — the others receive a conflict error.
4. The holder periodically renews the lease by updating `renewTime`.
5. If the holder fails to renew within `leaseDurationSeconds`, the lease is considered expired and another replica can acquire it.

The `leaderelection` package in `client-go` uses only local timestamps for leader determination. It does not trust the `renewTime` value stored in the lease record, which makes it tolerant to clock skew (though not to unbounded clock skew *rate*).

Implementation

The following Go example demonstrates setting up leader election using the `k8s.io/client-go/tools/leaderelection` package:

```
package main

import (
    "context"
    "os"
    "time"

    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/client-go/tools/leaderelection"
    "k8s.io/client-go/tools/leaderelection/resourcelock"
    "k8s.io/klog/v2"
)

func main() {
    // ... client setup omitted for brevity ...

    lock := &resourcelock.LeaseLock{
        LeaseMeta: metav1.ObjectMeta{
            Name:      "my-controller-lock",
            Namespace: "default",
        },
        Client: client.CoordinationV1(),
        LockConfig: resourcelock.ResourceLockConfig{
            Identity: hostname,
        },
    }

    leaderelection.RunOrDie(ctx, leaderelection.LeaderElectionConfig{
        Lock:            lock,
        LeaseDuration:   15 * time.Second,
        RenewDeadline:   10 * time.Second,
        RetryPeriod:     2 * time.Second,
        Callbacks: leaderelection.LeaderCallbacks{
            OnStartedLeading: func(ctx context.Context) {
                // Start the controller
                run(ctx)
            },
            OnStoppedLeading: func() {
                klog.Info("lost leadership")
                os.Exit(0)
            },
            OnNewLeader: func(identity string) {
                klog.Infof("new leader: %s", identity)
            },
        },
    })
}
```

The three timing parameters control leader election behavior:

- **LeaseDuration** (15s): how long a non-leader waits after observing a lease renewal before trying to acquire the lease.
- **RenewDeadline** (10s): how long the leader has to successfully renew before giving up.
- **RetryPeriod** (2s): how frequently candidates attempt to acquire or renew the lease.

The `kube-controller-manager` uses these same default values (15s/10s/2s) in production.

Required RBAC

The controller's ServiceAccount needs permissions to get, create, and update Lease objects:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: leader-election-role
rules:
- apiGroups: ["coordination.k8s.io"]
  resources: ["leases"]
  verbs: ["get", "create", "update"]
```

Bind this Role to the controller's ServiceAccount via a RoleBinding in the namespace where the Lease object lives.

Health Checking

The `leaderelection` package provides a `HealthzAdaptor` that integrates with the `/healthz` endpoint. If the leader fails to renew its lease but has not yet exited, the health check reports the process as unhealthy. This allows liveness probes to restart stuck leaders.

For a hands-on exercise implementing leader election with Lease objects, see Exercise 2: Implement Optimistic Locking and Leader Election via Lease.

References

- Kubernetes-Powered Leader Election in Go
- leaderelection package reference

Controller Sharding

Leader election provides high availability — if the active replica fails, a standby takes over. But it does not provide horizontal scaling: only one replica does work at any time. Controller sharding distributes reconciliation across multiple instances, each responsible for a subset of resources.

Beyond Leader Election

In a leader-elected controller with thousands of resources, a single reconciler becomes the

bottleneck. Sharding divides the resource space so that multiple replicas can reconcile concurrently without conflicting with each other. The challenge is assigning resources to shards consistently, rebalancing when shards are added or removed, and ensuring that owned child resources follow their parent's assignment.

Sharding via Labels

The kubernetes-controller-sharding project implements a label-based sharding mechanism:

- Each shard instance announces itself by maintaining a `Lease` object with a state label (`ready` or `dead`).
- A **sharder webhook** (`MutatingWebhookConfiguration`) intercepts `CREATE` and `UPDATE` requests for the sharded resource types.
- The webhook assigns a shard label to each object using **consistent hashing** — it hashes the object key and shard names onto a virtual ring, assigning the object to the nearest shard.
- Each controller instance watches only objects labeled with its own shard identity.
- Owned objects (those with `ownerReferences`) inherit their parent's shard assignment by hashing the owner's key instead of their own.

Scale-Down: Removing a Shard

When a shard instance is terminated:

1. The shard releases its Lease by clearing `holderIdentity`.
2. The sharder marks the shard as `dead`.
3. Orphaned objects have their shard label removed; the webhook reassigns them to remaining shards on the next update.

Scale-Up: Adding a Shard

When a new shard instance starts:

1. The new shard acquires a Lease and is marked `ready`.
2. The sharder identifies objects that should move to the new shard and adds a `drain` label.
3. The current shard stops reconciling drained objects, removes the drain and shard labels.
4. The webhook reassigns drained objects to the new shard.

This coordinated handoff prevents two shards from reconciling the same object simultaneously.

Case Study: OpenShift Ingress Controller Sharding

OpenShift supports running multiple Ingress Controllers, each responsible for a subset of Routes. This is a production example of sharding at the infrastructure level.

Sharding is configured via `routeSelector` and `namespaceSelector` on the `IngressController` resource:

```
apiVersion: operator.openshift.io/v1
kind: IngressController
metadata:
  name: internal
  namespace: openshift-ingress-operator
spec:
  routeSelector:
    matchLabels:
      type: internal
  domain: internal.example.com
```

OpenShift defines two sharding models:

- **Traditional (non-overlapping):** Each route is served by exactly one Ingress Controller. Shards are defined with mutually exclusive selectors.
- **Overlapped:** Routes can match multiple Ingress Controllers. This enables complex routing topologies but requires careful planning.

Use cases for ingress sharding include:

- Separating internal traffic from external traffic
- Isolating workloads that require different reliability guarantees
- Load-balancing across multiple router instances
- Implementing blue-green deployment strategies

Each Ingress Controller requires its own DNS entry. Routers do not forward unknown routes to other routers — if a route does not match any selector, it is unreachable.

When multiple Ingress Controllers serve the same Route (overlapped mode), each controller writes its own entry into the `status.ingress` array using atomic list types in the `routeIngress` status field. This prevents concurrent writers from corrupting each other's status entries.

References

- Controller Sharding Getting Started
- Ingress Sharding in OpenShift
- OpenShift Ingress Controller Sharding Case Study Notes

Week 3.1 Overview

This week takes us deeper into the kube-apiserver's extension points. We start with admission control webhooks for intercepting and modifying API requests before they reach storage. From there, we explore policy engines as a declarative alternative to hand-coded webhook logic. We then move into API discovery and the aggregation layer, examining how clients enumerate available resources and how custom API servers integrate with the control plane. Finally, we look at when CRDs are not enough and how to build a custom API server backed by non-etcd storage. Together, these topics cover the full spectrum of Kubernetes API-level extensibility.

Admission Control Webhooks

Admission Control in the Request Pipeline

Admission control runs after authentication and authorization but before the object is persisted to storage. It is the last line of defense for enforcing policy on write operations (create, update, delete). Reads such as `get`, `watch`, and `list` bypass admission control entirely.

The process has two phases:

1. **Mutating admission** — controllers in this phase may modify the incoming object (for example, injecting default values).
2. **Validating admission** — controllers in this phase accept or reject the request but cannot modify it.

If any controller in either phase rejects the request, the entire request fails immediately and an error is returned to the client.

For a refresher on where admission fits in the full request flow, see API Request Lifecycle.

Key Built-in Admission Controllers

Kubernetes compiles a set of admission controllers directly into the `kube-apiserver` binary. The most important ones to understand are:

- **NamespaceLifecycle** — prevents creation of objects in namespaces that are terminating or do not exist, and blocks deletion of the `default`, `kube-system`, and `kube-public` namespaces.
- **LimitRanger** — observes incoming requests and enforces the constraints defined in `LimitRange` objects within a namespace. It can also apply default resource requests to pods.
- **ServiceAccount** — automates service account setup for pods, including token injection and image pull secret references.
- **MutatingAdmissionWebhook** — calls external mutating webhooks that match the request. Matching webhooks run in serial and may modify the object.
- **ValidatingAdmissionWebhook** — calls external validating webhooks in parallel. If any webhook rejects, the request fails.
- **ResourceQuota** — enforces aggregate resource consumption limits defined in `ResourceQuota`

objects per namespace.

Writing a Webhook

A webhook is an HTTPS server that exposes an endpoint accepting an `AdmissionReview` request and returning an `AdmissionReview` response. The kube-apiserver sends a JSON payload describing the incoming object and operation, and the webhook replies with an `allowed: true` or `allowed: false` decision.

You register a webhook with the API server by creating a `ValidatingWebhookConfiguration` or `MutatingWebhookConfiguration` resource. Key fields in the configuration include:

- `rules` — defines which API groups, versions, resources, and operations (CREATE, UPDATE, DELETE) trigger the webhook.
- `clientConfig` — specifies how to reach the webhook: either a `service` reference (in-cluster) or a `url` (external endpoint), plus the CA bundle for TLS verification.
- `failurePolicy` — controls behavior when the webhook is unreachable. `Fail` rejects the request; `Ignore` allows it through.

The following example shows a `ValidatingWebhookConfiguration` that intercepts pod creation:

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: pod-validation-webhook
webhooks:
  - name: validate.pods.example.com
    rules:
      - apiGroups: [""]
        apiVersions: ["v1"]
        operations: ["CREATE"]
        resources: ["pods"]
    failurePolicy: Fail
    clientConfig:
      service:
        name: pod-validator
        namespace: webhooks
        path: /validate
      caBundle: <BASE64_ENCODED_CA>
    admissionReviewVersions: ["v1"]
    sideEffects: None
```

The OpenShift `generic-admission-server` library simplifies webhook development by reusing the `k8s.io/apiserver` infrastructure for TLS, authentication, and authorization. Rather than managing certificates manually, you deploy the webhook as an aggregated API server and configure the webhook to call back through the main API server endpoint.

Webhook chaining has important ordering implications. Mutating webhooks may

> run multiple times if a prior mutating webhook modifies the object, causing previously invoked webhooks to be re-invoked. The order between multiple webhooks of the same type is not guaranteed. Always set `failurePolicy: Fail` in production to avoid silently skipping validation when a webhook is unreachable.

Ready to build one yourself? See Exercise 3: Develop a Custom Validating Webhook.

References

- Admission Controllers Reference
- generic-admission-server

Policy Engines and Gatekeeper

Webhooks vs Policy Engines

Raw admission webhooks are powerful but operationally expensive. Each webhook requires writing Go code (or another language), building a container image, deploying and maintaining a service, and managing TLS certificates. When you need dozens of policies — all images from approved registries, all pods with resource limits, all namespaces with a contact label — the overhead multiplies quickly.

Policy engines solve this by providing a declarative way to define admission policies using CRDs. Instead of deploying a new webhook for each rule, you deploy the engine once and express individual policies as Kubernetes objects.

OPA Gatekeeper

OPA Gatekeeper v3 integrates the Open Policy Agent with Kubernetes using the OPA Constraint Framework. The architecture centers on two CRD types:

ConstraintTemplates define the policy logic written in Rego (OPA's query language) along with a parameter schema. A template is reusable — it describes *what* to check and *what parameters* can be customized.

Constraints instantiate a template with specific parameters and a `match` block that scopes enforcement to particular resource kinds.

The following example defines a template that requires specific labels, then creates a constraint enforcing it on namespaces:

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8srequiredlabels
spec:
  crd:
    spec:
```

```
      names:
        kind: K8sRequiredLabels
      validation:
        openAPIV3Schema:
          properties:
            labels:
              type: array
              items:
                type: string
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8srequiredlabels

        deny[{"msg": msg, "details": {"missing_labels": missing}}] {
          provided := {label | input.review.object.metadata.labels[label]}
          required := {label | label := input.parameters.labels[_]}
          missing := required - provided
          count(missing) > 0
          msg := sprintf("you must provide labels: %v", [missing])
        }
```

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: ns-must-have-owner
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Namespace"]
  parameters:
    labels: ["owner"]
```

Audit

Gatekeeper does not only evaluate new requests. Its audit functionality periodically scans existing resources in the cluster and checks them against enforced constraints. Violations are recorded in the `status` field of the corresponding Constraint object, making it easy to detect pre-existing misconfigurations that were created before a policy was applied.

Data Replication

Some policies require cross-resource visibility. For example, enforcing globally unique ingress hostnames means the policy engine needs access to all Ingress objects, not just the one being admitted. Gatekeeper supports data replication via a `Config` CRD that specifies which resource types should be cached into OPA for use in Rego rules.

ValidatingAdmissionPolicy (CEL-based)

Starting in Kubernetes v1.30, `ValidatingAdmissionPolicy` reached stable status as a built-in alternative to external webhook infrastructure. Instead of calling out to an HTTP server, policies are expressed as CEL (Common Expression Language) expressions evaluated directly inside the API server.

This eliminates the need for a separate webhook deployment, TLS certificates, and network round-trips. CEL-based policies are best suited for straightforward validation rules — such as checking label presence or resource limits — where the full power of Rego or Go code is not needed.

Use **Gatekeeper** when you need complex cross-resource policies, audit of existing resources, or Rego's full expressiveness. Use **ValidatingAdmissionPolicy** for simpler, latency-sensitive validations that benefit from running in-process within the API server.

References

- OPA Gatekeeper: Policy and Governance for Kubernetes

API Discovery and Aggregation

API Discovery

Before a client can interact with the Kubernetes API, it needs to know what resources are available and which operations they support. The discovery API provides this information.

Every Kubernetes cluster publishes discovery data at well-known endpoints:

- `/api` — lists the core API group (resources like Pods, Services, ConfigMaps).
- `/apis` — lists all named API groups (apps, batch, networking.k8s.io, and any custom groups).
- `/apis/<group>/<version>` — describes the resources available within a specific group version, including supported verbs, scope (cluster or namespace), and subresources.

For each resource, the discovery response includes the resource name, whether it is namespaced, the supported verbs (get, list, watch, create, update, patch, delete), and its group-version-kind.

Additionally, OpenAPI schema documents are available at `/openapi/v2` and `/openapi/v3`, providing full structural definitions for every resource type. OpenAPI v3 is the preferred format because it provides a lossless representation of the API schema.

The Discovery Storm Problem

Traditional (unaggregated) discovery requires clients to make one HTTP request per API group version to enumerate its resources. On a cluster with N API groups, a client needs at least N+1 requests just to build a complete picture of available APIs. With CRDs and aggregated API servers, a production cluster can easily have 80 or more group versions.

This creates a **discovery storm** — a burst of requests every time a client like `kubectl` initializes or its cache expires. The `kubectl` client mitigated this with a 6-hour cache, but that introduced staleness problems, and the full refresh after expiry still generated a storm. On large OpenShift clusters with many CRDs and operators, the discovery storm was a significant performance and throttling issue.

Aggregated Discovery (KEP-3352)

Aggregated Discovery, stable since Kubernetes v1.30, collapses all discovery information into just 2 requests — one to `/api` and one to `/apis` — by using content negotiation. Clients request the aggregated format with the header:

```
Accept: application/json;v=v2;g=apidiscovery.k8s.io;as=APIGroupDiscoveryList
```

The server returns an `APIGroupDiscoveryList` containing every group, version, and resource in a single response. ETag headers enable efficient cache validation: if the discovery data has not changed, the server returns a `304 Not Modified` with no body.

Clients using `client-go` v1.26 and later automatically use aggregated discovery when the server supports it, with a transparent fallback to unaggregated discovery for older clusters.

CRDs vs API Aggregation

Kubernetes provides two mechanisms for adding custom resources: CustomResourceDefinitions (CRDs) and aggregated API servers. They serve different needs:

Aspect	CRDs	Aggregated API Server
Setup	Declarative YAML, no coding required	Requires building and deploying a Go binary
Storage	etcd, managed by the kube-apiserver	Custom backend (any database, external service)
Validation	OpenAPI v3 schema + CEL expressions	Full Go validation logic, arbitrary complexity
Subresources	`/status` and `/scale` only	Any custom subresource (e.g., `/logs`, `/exec`)
Protocol Buffers	Not supported	Supported
Best for	Most extension use cases	Non-etcd storage, complex multi-version conversion, protocol buffer support

For the majority of use cases, CRDs are the right choice. Reach for an aggregated API server only when you need a non-etcd storage backend, custom subresources beyond `/status` and `/scale`, or protocol buffer support.

References

- The Kubernetes API
- KEP-3352: Aggregated Discovery
- Custom Resources

Custom API Servers

When CRDs Are Not Enough

CRDs handle the vast majority of Kubernetes extension use cases, but there are scenarios where their limitations become blockers:

- **Non-etcd storage** — your data lives in a relational database, a time-series store like Prometheus, or an external catalog service. CRDs always store objects in etcd; there is no way to redirect them.
- **Day-zero validation** — validations that must be enforced before the first object is ever created. With CRDs, you create the CRD, then deploy a validating webhook. There is a window where objects can be created without validation. A custom API server has validation compiled in from the start.
- **Complex subresources** — CRDs support only `/status` and `/scale`. If you need custom subresources like `/logs`, `/exec`, or `/proxy`, you need a custom API server.
- **Protocol buffer support** — CRDs serve only JSON. If your clients need protobuf serialization for performance, only an aggregated API server can provide it.

The API Aggregation Layer

The aggregation layer lets you register a custom API server so that the kube-apiserver acts as a reverse proxy for your API group. Clients see a unified API surface — requests to your custom group are transparently forwarded to your server.

Registration is done by creating an `APIService` object:

```
apiVersion: apiregistration.k8s.io/v1
kind: APIService
metadata:
  name: v1.packages.operators.coreos.com
spec:
  group: packages.operators.coreos.com
  version: v1
  service:
    name: packageserver
    namespace: openshift-operator-lifecycle-manager
  groupPriorityMinimum: 2000
  versionPriority: 15
  caBundle: <BASE64_ENCODED_CA>
```

When a client sends a request to `/apis/packages.operators.coreos.com/v1/⋯`, the kube-apiserver forwards it to the `packageserver` service. Authentication, authorization, and audit logging are handled by the kube-apiserver before forwarding.

Building with k8s.io/apiserver

The `k8s.io/apiserver` library provides the generic framework for building a Kubernetes-compatible API server. It handles TLS termination, authentication (delegated to the main API server via `TokenReview`), authorization (via `SubjectAccessReview`), admission control, OpenAPI schema generation, and the storage interface.

The kubernetes/sample-apiserver repository is the reference implementation. The typical workflow is:

1. Define your API types in `pkg/apis/<group>/types.go`.
2. Run code generation (`hack/update-codegen.sh`) to produce deep-copy functions, client sets, listers, and informers.
3. Implement a storage backend that satisfies the `rest.Storage` interface.
4. Wire everything together in a server binary that calls `GenericAPIServer.InstallAPIGroup()`.

The OpenShift generic-admission-server library builds on this same foundation, providing a streamlined path for admission webhooks deployed as aggregated API servers.

Case Study: OLM Package Server

The Operator Lifecycle Manager (OLM) provides a concrete example of why a custom API server is necessary.

OLM needs to serve **PackageManifest** resources — metadata about operators available for installation — sourced from gRPC catalog services. In a large OpenShift cluster, there can be 531 operators across 74 catalog sources and namespaces. If each PackageManifest were stored as a CRD object, that would create approximately 39,000 etcd objects (531 operators multiplied by 74 namespace-scoped copies).

Instead, the OLM package server is an aggregated API server that reads directly from the gRPC catalog sources at request time. No data is persisted in etcd. The package manifests are computed on the fly, eliminating both the storage overhead and the synchronization complexity of keeping CRD objects in sync with external catalogs.

Other OpenShift components follow the same pattern:

- The **monitoring API server** reads metrics data from Prometheus rather than storing it in etcd.
- The **OAuth API server** manages OAuth tokens with specialized storage and lifecycle requirements that go beyond what CRDs offer.

Before committing to a custom API server, consider the operational cost: you must build, deploy, and maintain a separate binary; handle upgrades and version skew with the kube-apiserver; and implement health checks so the aggregation layer

> can detect failures. Choose this path only when the technical requirements genuinely demand it.

References

- Choosing a Method for Adding Custom Resources
- API Aggregation Layer
- sample-apiserver
- package-server-manager

Week 3.2 — The Other Stuff

This week covers the other important topics that complement the operator and controller focus of the course — networking (Ingress and Gateway API), service mesh, authentication and authorization, and cluster security hardening.

- **Ingress and Gateway API** — how external traffic reaches your services, from the original Ingress resource to its role-oriented successor.
- **Service Mesh Fundamentals** — what a service mesh solves, its architecture, and how Istio implements traffic management, security, and observability.
- **Authentication and Authorization** — how Kubernetes identifies users and decides what they can do, including the Node authorizer and emerging ReBAC approaches.
- **Security Context and Hardening** — locking down pods with SecurityContext fields, the Security Profiles Operator, and OpenShift SecurityContextConstraints.

Ingress and Gateway API

Ingress: the original HTTP routing API

The `Ingress` resource (stable since Kubernetes v1.19) exposes HTTP and HTTPS routes from outside the cluster to Services inside it. An Ingress controller — typically NGINX, HAProxy, or a cloud load balancer — watches Ingress objects and configures the underlying data plane accordingly.

Key concepts you should know:

- **IngressClass** — selects which controller handles a given Ingress. If `ingressClassName` is omitted, the cluster's default IngressClass is used.
- **Path types** — `Exact` matches a path literally, `Prefix` matches by path prefix, and `ImplementationSpecific` defers the interpretation to the controller.
- **TLS termination** — an Ingress can terminate TLS using a Secret that contains the certificate and private key, offloading encryption from backend pods.
- **Default backend** — a catch-all Service that handles requests that match no rule.

A minimal Ingress looks like this:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
spec:
  ingressClassName: nginx-example
  rules:
  - http:
      paths:
      - path: /testpath
        pathType: Prefix
```

```
        backend:
          service:
            name: test
            port:
              number: 80
```

The Ingress API has been frozen. The Kubernetes project recommends the Gateway API for all new work and will make no further changes to the Ingress spec.

Gateway API: the role-oriented successor

The Gateway API is a family of CRDs that provide dynamic infrastructure provisioning and advanced traffic routing. It was designed around four principles — role-oriented, portable, expressive, and extensible — and has four stable kinds: `GatewayClass`, `Gateway`, `HTTPRoute`, and `GRPCRoute`.

Role-oriented resource model

The key improvement over Ingress is the separation of concerns across organizational roles:

1. **Infrastructure provider** manages `GatewayClass` — defines the controller implementation (analogous to a StorageClass for volumes).
2. **Cluster operator** manages `Gateway` — provisions an instance of the load balancer, configures listeners, ports, and TLS.
3. **Application developer** manages `HTTPRoute` / `GRPCRoute` — defines routing rules that attach to a Gateway.

This separation means platform teams can manage load balancers while developers manage their own application routing independently.

Gateway + HTTPRoute example

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: example-gateway
spec:
  gatewayClassName: example-class
  listeners:
  - name: http
    protocol: HTTP
    port: 80
    hostname: "www.example.com"
    allowedRoutes:
      namespaces:
        from: Same
---
```

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: example-httproute
spec:
  parentRefs:
  - name: example-gateway
  hostnames:
  - "www.example.com"
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /login
    backendRefs:
    - name: example-svc
      port: 8080
```

By default a Gateway only accepts Routes from the same namespace. Cross-namespace routing requires configuring `allowedRoutes` on the listener.

Migrating from Ingress

The Gateway API does not include an Ingress kind, so migration is a one-time conversion. The upstream migration guide walks through the process step by step.

For more on how ingress controllers can be sharded across nodes in large clusters, see Week 2.2 on controller sharding.

References

- Ingress
- Gateway API

Service Mesh Fundamentals

What a service mesh solves

In a microservices architecture every service needs service discovery, load balancing, retries, timeouts, mTLS, and telemetry. Implementing those cross-cutting concerns inside each application — across multiple languages and frameworks — is tedious and error-prone. A service mesh moves all of that logic into the infrastructure layer so that applications simply send and receive traffic on localhost without knowing about the network topology.

The capabilities of a service mesh fall into three categories:

- **Traffic management** — dynamic service discovery, routing, traffic splitting for canary releases and A/B tests, retries, timeouts, circuit breaking, and rate limiting.

- **Security** — automatic mutual TLS (mTLS) for all service-to-service communication, certificate rotation, and fine-grained access policies based on service identity.
- **Observability** — uniform collection of golden-signal metrics (latency, traffic, errors, saturation), distributed traces, and access logs across every hop in the mesh.

Architecture: data plane + control plane

A service mesh consists of two layers:

- **Data plane** — lightweight network proxies (typically Envoy) deployed alongside each service instance. In the traditional sidecar model every pod gets its own proxy container. The proxies intercept all inbound and outbound traffic, apply routing rules, enforce mTLS, and emit telemetry.
- **Control plane** — a centralized component that configures the proxies at runtime. It converts high-level routing rules and security policies into proxy-specific configuration and distributes certificates for mTLS.

Istio: the most widely adopted mesh

Istio is a CNCF graduated project originally created by Google, IBM, and Lyft. Its control plane is a single binary called `istiod` that consolidates the earlier Pilot, Citadel, and Galley components. Istiod handles service discovery, certificate generation, and configuration distribution to the Envoy sidecars.

Istio provides two types of authentication:

- **Peer authentication** — service-to-service mTLS, enforced via a `PeerAuthentication` policy.
- **Request authentication** — end-user authentication using JWT validation or OIDC providers.

Traffic splitting with VirtualService

Istio uses CRDs — `VirtualService` and `DestinationRule` — to control traffic routing. The following example sends 90% of traffic to v1 of a service and 10% to v2:

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: shipping-service
spec:
  hosts:
  - shipping-service
  http:
  - route:
    - destination:
        host: shipping-service
        subset: v1
      weight: 90
    - destination:
```

```
        host: shipping-service
        subset: v2
      weight: 10
---
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: shipping-service
spec:
  host: shipping-service
  subsets:
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
```

Ambient mode: sidecar-less mesh

Istio's newer **ambient mode** eliminates the sidecar container entirely. Instead, a per-node `ztunnel` proxy handles L4 concerns (mTLS, basic telemetry) for all pods on the node, while optional `waypoint` proxies handle L7 policies when needed. Ambient mode reduces resource overhead and simplifies the pod lifecycle because there is no longer a sidecar to inject, restart, or drain.

A service mesh adds real operational complexity. Evaluate whether your application genuinely needs mesh-level traffic management, mTLS, and observability before adopting one — for simpler deployments, Kubernetes-native NetworkPolicies and Gateway API may be sufficient.

References

- Service Mesh Architecture with Istio
- The Istio Service Mesh
- GAMMA: Gateway API for Service Mesh

Authentication and Authorization

Every request to the Kubernetes API server passes through authentication, then authorization, before it reaches admission control and storage. Understanding how these layers work is essential for building controllers and operators that interact with the API safely. For the full request flow, see API Request Lifecycle.

Authentication: who are you?

Kubernetes has two categories of identity:

- **Service accounts** — managed by the Kubernetes API, bound to namespaces, and backed by

projected token volumes or Secrets. Every pod automatically gets a service account token mounted at `/var/run/secrets/kubernetes.io/serviceaccount/token`.

- **Normal users** — Kubernetes has no `User` object. Identity comes entirely from the authentication layer: a valid client certificate, a bearer token, or an OIDC token.

The API server evaluates authenticator plugins in order. The first plugin that succeeds short-circuits the chain and populates a `UserInfo` struct with username, UID, groups, and optional extra fields. Available authentication methods include:

- **X.509 client certificates** — any certificate signed by the cluster CA is accepted. The `commonName` field becomes the username; `organization` fields become groups.
- **Bearer tokens** — static token files or bootstrap tokens, primarily used during cluster setup.
- **OIDC tokens** — the recommended approach for human users. The API server validates JWT tokens against a configured OIDC provider without acting as an identity provider itself.
- **Service account tokens** — projected, time-limited JWTs issued by the API server for in-cluster workloads.
- **Webhook token review** — delegates authentication to an external service via the `TokenReview` API.

Requests that do not match any authenticator are treated as anonymous (`system:anonymous` user, `system:unauthenticated` group).

Authorization: what can you do?

After authentication, the API server consults a chain of authorizer modules. Each module returns `Allow`, `Deny`, or `NoOpinion`. An `Allow` or `Deny` short-circuits the chain; `NoOpinion` passes the decision to the next module.

RBAC

Role-Based Access Control is the most common authorizer. `Role` and `ClusterRole` objects define sets of allowed verbs on API resources; `RoleBinding` and `ClusterRoleBinding` map those roles to users, groups, or service accounts. RBAC is purely additive — it never returns `Deny`, only `Allow` or `NoOpinion`.

Node authorizer

The Node authorization mode exists specifically for kubelets. A kubelet authenticates as `system:node:<nodeName>` and the Node authorizer grants it access only to the pods, secrets, configmaps, and persistent volumes that are bound to its own node. This graph-based decision prevents a compromised node from reading secrets belonging to pods on other nodes.

The logic follows a resource graph:

```
node <- pod
node <- pod <- secret
node <- pod <- configmap
node <- pod <- pvc <- pv <- secret
```

For resources outside this graph — such as listing all secrets cluster-wide — the Node authorizer returns `NoOpinion` and the request falls through to RBAC. The `NodeRestriction` admission controller further limits what properties of `Node` and `Pod` objects a kubelet can modify.

The `kubectl auth can-i --list` command only reports RBAC-granted permissions. Rights granted by the Node authorizer will not appear in that output, which can be misleading during security audits.

Webhook authorization

For requirements that exceed RBAC's additive model, Kubernetes supports external webhook authorizers. The API server sends a `SubjectAccessReview` to the webhook, which can return `Allow`, `Deny`, or `NoOpinion` — including native deny rules that RBAC cannot express.

ReBAC: relation-based access control

ReBAC is an emerging approach inspired by the Google Zanzibar paper. Instead of flat role-to-permission mappings, authorization state is modelled as a graph with typed nodes and directed relations — for example, `user:alice  editor → document:secret`.

The kube-rebac-authorizer prototype demonstrates how ReBAC (built on the CNCF project OpenFGA) can replace both the RBAC and Node authorizers with a single graph-based engine. Key advantages over RBAC include:

- **Native deny rules** — the RBAC authorizer is additive-only, but ReBAC can express deny decisions directly.
- **Inheritance** — grant access to a Deployment and automatically propagate it to its child ReplicaSets and grandchild Pods without separate role entries.
- **Cross-system integration** — external permission sources (LDAP groups, cloud IAM roles) can be projected into the same graph as Kubernetes RBAC objects.

ReBAC is still experimental in the Kubernetes ecosystem, but it signals the direction that SIG-Auth is exploring for fine-grained, graph-aware authorization.

References

- Authenticating
- Let's Talk About Kubelet Authorization
- kube-rebac-authorizer

Security Context and Hardening

A `SecurityContext` defines privilege and access control settings for a pod or container. It is the primary mechanism for hardening workloads at the pod spec level.

Pod-level vs container-level SecurityContext

Kubernetes supports security context at two scopes:

- **Pod-level** (`spec.securityContext`) — applies to all containers in the pod. Fields like `runAsUser`, `runAsGroup`, `fsGroup`, and `supplementalGroups` are set here.
- **Container-level** (`spec.containers[].securityContext`) — overrides pod-level settings for a specific container. Fields like `allowPrivilegeEscalation`, `capabilities`, `readOnlyRootFilesystem`, and `seccompProfile` are typically set here.

When both levels specify the same field, the container-level value takes precedence.

Key SecurityContext fields

Field	Purpose
`runAsUser` / `runAsGroup`	Sets the UID and primary GID for all processes in the container.
`runAsNonRoot`	Rejects the pod at admission if the container image would run as UID 0.
`readOnlyRootFilesystem`	Mounts the container's root filesystem as read-only, preventing writes outside of explicitly mounted volumes.
`allowPrivilegeEscalation`	Controls whether a child process can gain more privileges than its parent. Setting this to `false` sets the `no_new_privs` kernel flag.
`capabilities`	Adds or drops Linux capabilities. A hardened container drops `ALL` and adds back only what it needs.
`seccompProfile`	Selects a seccomp profile that filters which syscalls the process can make. `RuntimeDefault` is the baseline.

A minimal hardened pod

The following pod spec demonstrates a defense-in-depth configuration that is compatible with most workloads:

```
apiVersion: v1
kind: Pod
metadata:
  name: hardened-pod
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
    seccompProfile:
      type: RuntimeDefault
  containers:
```

```
  - name: app
    image: registry.example.com/app:v1
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop: ["ALL"]
      readOnlyRootFilesystem: true
```

This configuration ensures the container cannot run as root, cannot escalate privileges, cannot write to its root filesystem, and is restricted to the runtime's default set of allowed syscalls.

Security Profiles Operator

Managing seccomp, SELinux, and AppArmor profiles manually across nodes is operationally painful. The Security Profiles Operator (SPO) is a Kubernetes-sigs project that automates profile lifecycle management.

SPO provides CRDs for each profile type:

- `SeccompProfile` — declares a seccomp filter and distributes it to nodes.
- `SELinuxProfile` — defines and installs SELinux policies.
- `AppArmorProfile` — manages AppArmor profiles on nodes that support them.

Two additional features make SPO especially useful:

- **Profile recording** — SPO can observe a running workload (via audit logs or eBPF) and generate a least-privilege profile automatically.
- **Profile binding** — a `ProfileBinding` resource links a profile to a container image, so the profile is applied automatically whenever that image is deployed.

OpenShift SecurityContextConstraints

OpenShift extends the upstream Pod Security model with **SecurityContextConstraints** (SCCs) — cluster-scoped policies that define what security contexts a pod is allowed to use.

SCCs are evaluated by an admission controller at pod creation time. The controller checks the pod's service account against the available SCCs and rejects the pod if its requested security context violates the constraints.

OpenShift ships with several default SCCs:

- `restricted-v2` — the default for most workloads. Prevents running as root, drops all capabilities, and requires a seccomp profile.
- `anyuid` — allows the container to run as any UID, useful for legacy images that require root.
- `privileged` — unrestricted access, reserved for infrastructure components like CNI plugins and storage drivers.

Granting the `privileged` SCC to a workload effectively disables all pod-level

> security controls. Use it only for infrastructure components that genuinely require host-level access, and always through a dedicated service account.

References

- Configure a Security Context
- Security Profiles Operator

Week 4.1 — All About kube-scheduler

The kube-scheduler is at the heart of workload placement. This week examines how the scheduler selects nodes for pods, how resource requests and limits affect scheduling and runtime behavior, and how Dynamic Resource Allocation extends scheduling to non-fungible resources like GPUs.

- How the Scheduler Works — the filtering, scoring, and binding pipeline that turns a pending pod into a running workload.
- Resource Management — requests, limits, QoS classes, and the kernel enforcement mechanisms behind them.
- Dynamic Resource Allocation — the claim-based model for scheduling specialized hardware that CPU and memory numbers alone cannot describe.

How the Scheduler Works

What the scheduler does

The kube-scheduler is a controller that watches for Pods with no `.spec.nodeName` and assigns them to nodes. Its core loop is deceptively simple: pop a pod off the queue, find the best node, write a Binding object back to the API server. The kubelet on the chosen node then takes over and actually starts the containers.

Internally the scheduler uses an informer to populate a priority queue of unscheduled pods. When new pods appear, an event handler pushes them onto the queue. The scheduler pulls pods one at a time and runs the full scheduling cycle for each.

The scheduling cycle

Every pod passes through three phases:

1. **Filtering (predicates)** — the scheduler evaluates every node to determine which ones *can* run the pod. Nodes that lack sufficient CPU, violate taints and tolerations, or fail node-affinity rules are eliminated. If no node survives filtering, the scheduler moves to the PostFilter stage to attempt preemption.
2. **Scoring (priorities)** — the remaining feasible nodes are ranked. Scoring functions consider factors like balanced resource utilization, pod spreading across zones, and inter-pod affinity preferences. Each scoring plugin returns a value, and the framework sums them to produce a final rank per node.
3. **Binding** — the scheduler assigns the pod to the highest-scoring node by creating a Binding object in the API server. Binding runs asynchronously so the main scheduling loop is not blocked, which increases throughput.

Scheduling Framework extension points

The modern scheduler is built on the *Scheduling Framework*, a plugin architecture where every built-in feature — resource fit checks, taint toleration, node affinity — is implemented as a plugin.

The framework exposes the following extension points, executed in order:

Extension Point	Purpose
QueueSort	Determines the order pods are dequeued for scheduling (default: by priority).
PreFilter	Initializes per-cycle plugin state before filtering begins.
Filter	Decides whether a node is feasible for the pod (replaces the old predicate functions).
PostFilter	Runs when *no* node passes filtering — typically triggers preemption.
PreScore	Prepares state needed by scoring plugins.
Score	Ranks each feasible node (replaces the old priority functions).
Reserve	Claims resources on the chosen node optimistically before binding.
Permit	Gates binding — a plugin can approve, deny, or ask the pod to wait.
PreBind	Runs setup that must succeed before the bind (e.g., provisioning a volume).
Bind	Writes the Binding object to the API server.
PostBind	Runs cleanup or notification logic after a successful bind.

Because native features use the same plugin API, custom scheduler plugins can introduce new constraints or scoring logic without forking the scheduler.

Preemption

When the Filter stage finds no feasible node, the PostFilter extension point fires. The default implementation — `DefaultPreemption` — searches for a node where evicting lower-priority pods would make room for the pending pod.

The algorithm works roughly like this:

1. For each node, remove all pods with a lower priority than the pending pod and check whether the pod now passes all Filter plugins.
2. Try to add back as many removed pods as possible (preferring PDB-protected pods and higher-priority victims) while still allowing the pending pod to schedule.
3. The minimal remaining set of pods that must be evicted becomes the *eviction candidate* for that node.
4. The best candidate across all nodes is selected — preferring fewer PDB violations, lower total victim priority, and less churn.
5. The victim pods are deleted, and the pending pod is re-queued to schedule once the victims terminate.

The scheduler also takes a snapshot of cluster state at the start of each cycle so that all plugins see a consistent view, even as the real cluster changes under them.

Why the scheduler does not resync

Most Kubernetes controllers periodically resync their informers to recover from missed events. The scheduler intentionally sets its resync period to zero. As the maintainers explained when a resync was proposed, they prefer correctness bugs to be surfaced and fixed rather than silently masked by a periodic full re-list. If a pod gets stuck in `Pending` because an event was lost, the fix belongs in the event-handling code, not in a retry band-aid.

Complementary tools

The kube-scheduler only acts at pod-creation time — it never moves a running pod. Several companion tools fill the gaps:

- **Descheduler** — an add-on that identifies pods whose placement has become suboptimal (e.g., under-utilized or over-loaded nodes) and evicts them so the scheduler can place them again.
- **Horizontal Pod Autoscaler (HPA)** — adjusts the number of pod replicas based on observed CPU, memory, or custom metrics.
- **Vertical Pod Autoscaler (VPA)** — adjusts resource requests and limits on existing pods to match actual usage.
- **Cluster Autoscaler** — provisions or removes nodes when the scheduler cannot place pods (or when nodes are persistently idle).

Together, these tools close the feedback loop that the scheduler alone cannot provide.

References

- How Does the Kubernetes Scheduler Work?
- A Deeper Dive of kube-scheduler

Resource Management

Requests vs. limits

Every container in a Pod can declare two resource values: a **request** and a **limit**. They serve fundamentally different purposes:

- **Requests** tell the kube-scheduler how much capacity to reserve. The scheduler sums the requests of all containers in a Pod and only places it on a node that has enough unallocated resources. Requests are a scheduling-time concept — they do not constrain the running container.
- **Limits** tell the kubelet (and ultimately the Linux kernel) how much a container is allowed to consume at runtime. A container can burst above its request when spare capacity exists, but it cannot exceed its limit.

If you set a limit without a request, Kubernetes copies the limit value into the request automatically.

CPU

CPU is measured in *millicores* (symbol `m`). One core equals `1000m`. The expression `0.5` is equivalent to `500m` — half a core.

CPU limits are enforced through CFS (Completely Fair Scheduler) throttling. When a container reaches its CPU limit, the kernel restricts its CPU time for the remainder of the scheduling interval. The container is **not** killed — it simply runs more slowly. Because throttling is invisible to many monitoring tools, a sluggish application with low reported CPU usage is a classic sign that the CPU limit is set too low.

Memory

Memory is measured in bytes. Kubernetes accepts suffixes like `Mi` (mebibytes) and `Gi` (gibibytes).

Memory limits are enforced by the kernel's OOM (Out-Of-Memory) killer. When a container allocates more memory than its limit allows and the system is under pressure, the kernel terminates it. Unlike CPU throttling, an OOM kill is destructive — the container restarts (subject to `restartPolicy`), and in-flight work is lost.

Example Pod with requests and limits

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-demo
spec:
  containers:
  - name: app
    image: nginx:1.27
    resources:
      requests:
        cpu: "250m"
        memory: "64Mi"
      limits:
        cpu: "500m"
        memory: "128Mi"
```

The scheduler looks at the request — 250m CPU and 64Mi memory — to decide where this Pod can fit. Once running, the kernel enforces the limit — at most 500m CPU and 128Mi memory.

QoS classes

Kubernetes automatically assigns each Pod a Quality of Service class based on its resource declarations. The QoS class determines eviction priority when a node runs out of resources.

QoS Class	Condition	Eviction Priority
Guaranteed	Every container sets requests == limits for both CPU and memory.	Evicted last.
Burstable	At least one container has a request or limit, but they are not all equal.	Evicted after BestEffort pods.
BestEffort	No container sets any request or limit.	Evicted first.

Setting requests equal to limits for production workloads gives them `Guaranteed` QoS and protects them from eviction during memory pressure.

Namespace-level guardrails

Cluster administrators can enforce resource policies at the namespace level with two objects:

- **LimitRange** — sets default requests and limits for containers that omit them, and defines per-container minimums and maximums. This prevents a single container from requesting an unreasonable amount of resources.
- **ResourceQuota** — caps the total amount of CPU, memory, and object count (pods, services, etc.) that a namespace can consume. Once a quota is exhausted, new pods in that namespace will fail to create until existing resources are freed.

Together, LimitRange and ResourceQuota let platform teams give development teams self-service access to a cluster without risking a single team monopolizing shared resources.

Practical debugging tips

- A pod stuck in `Pending` with a `FailedScheduling` event usually means its requests exceed available node capacity. Run `kubectl describe node` and compare *Allocatable* with *Allocated resources*.
- A container restarting with exit reason `OOMKilled` needs a higher memory limit — or a memory leak investigation.
- High `container_cpu_cfs_throttled_seconds_total` in Prometheus indicates CPU throttling. Consider raising the CPU limit or removing it entirely if the node has spare capacity.

References

- Resource Management for Pods and Containers

Dynamic Resource Allocation

The problem DRA solves

Traditional Kubernetes scheduling treats resources as fungible quantities — 500m of CPU on node A is the same as 500m on node B. That model breaks down for specialized hardware like GPUs, FPGAs, and high-performance network interfaces. A workload may need "one NVIDIA A100 with 80

GB VRAM", not just "some amount of GPU". These resources are non-fungible: they differ in type, capability, and availability across nodes.

The older Device Plugin framework handled simple pass-through allocation, but it could not express device attributes, share devices across containers, or let the scheduler reason about which *specific* device to assign. Dynamic Resource Allocation (DRA) replaces that model with a claim-based approach inspired by PersistentVolumeClaims.

Key API resources

DRA is part of the `resource.k8s.io/v1` API group and introduces three core resources:

DeviceClass

Defines a category of devices available in the cluster. A DeviceClass is created by device drivers or cluster administrators and tells the scheduler what kind of hardware exists and how to select it. For example, a driver might publish a `gpu-class` that covers all GPU models it manages.

ResourceClaim

A request for access to one or more devices. A ResourceClaim references a DeviceClass and can use CEL selectors to filter for specific device attributes — model, VRAM size, driver version, and so on. The scheduler coordinates with the DRA driver to find a node that has a matching device and allocates it to the claim.

ResourceClaimTemplate

A template that Kubernetes uses to generate a per-pod ResourceClaim automatically. This is analogous to how a VolumeClaimTemplate works in a StatefulSet: each pod gets its own claim, and the claim is deleted when the pod terminates.

A fourth resource, **ResourceSlice**, is created by the device driver on each node. ResourceSlices advertise what devices are available and their attributes, giving the scheduler the information it needs to match claims to nodes.

How scheduling works with DRA

When a pod references a ResourceClaim (directly or via a ResourceClaimTemplate), the scheduling cycle adds an extra coordination step:

1. The scheduler identifies nodes that might satisfy the claim based on published ResourceSlices.
2. For each candidate node, the scheduler asks the DRA driver whether the claim can be allocated on that node.
3. Nodes where allocation succeeds are considered feasible; others are filtered out.
4. The pod is scored and bound as usual, and the allocated device information is recorded in the ResourceClaim status.

This means DRA extends the scheduler's Filter stage with device-aware logic without requiring changes to the scheduler core.

Prioritized alternatives

A ResourceClaim can specify a **prioritized list** of sub-requests using `firstAvailable`. The scheduler evaluates each alternative in order and picks the first one that can be satisfied. This lets you express preferences — for example, prefer a large GPU but fall back to two smaller ones:

```
apiVersion: resource.k8s.io/v1
kind: ResourceClaimTemplate
metadata:
  name: gpu-claim-template
spec:
  spec:
    devices:
      requests:
      - name: gpu-req
        firstAvailable:
        - name: large-gpu
          deviceClassName: gpu-class
          selectors:
          - cel:
              expression: device.attributes["gpu-driver.example.com"].vram >= 80
        - name: small-gpus
          deviceClassName: gpu-class
          selectors:
          - cel:
              expression: device.attributes["gpu-driver.example.com"].vram >= 24
          count: 2
```

When scoring nodes, the scheduler prefers nodes that can satisfy a higher-ranked alternative, so the large GPU is chosen when available.

Device sharing

Unlike the old Device Plugin model, DRA supports sharing a device across multiple containers or even multiple pods. If two containers in the same pod reference the same ResourceClaim, they both get access to the allocated device. Separate pods can also share a device by referencing the same named ResourceClaim, as long as the claim exists in the same namespace.

Relationship to traditional resources

DRA does not replace CPU and memory requests. A pod that needs a GPU still needs CPU and memory to run the rest of its code. In practice, pods that use DRA set both traditional resource requests *and* ResourceClaims:

- The scheduler uses CPU and memory requests to ensure the node has enough compute.
- The scheduler uses the ResourceClaim to ensure the node has the right device.
- Only nodes that satisfy both constraints are feasible.

DRA is a rapidly evolving area. Some features like workload-level claims and device taints are still in alpha or beta. Check the upstream DRA documentation for the latest API stability and feature gates.

References

- Dynamic Resource Allocation

Week 4.2 — Ubiquitous Topics

This final week zooms out to the broader operator ecosystem and alternative Kubernetes form factors. We examine the operator capability model and framework landscape, then survey control plane architectures that diverge from the standard self-hosted model.

- **The Operator Landscape** — the formal definition of operators, the five-level capability model, major frameworks, and proven design patterns.
- **Alternative Control Planes and Form Factors** — hosted control planes, Kubernetes-like API platforms, and minimal distributions built for edge and resource-constrained environments.

The Operator Landscape

The operator concept has been formalised through years of building controllers for stateful workloads. Below you will find a precise definition, a maturity model, the major frameworks, and the design patterns that keep operators maintainable at scale.

What is an Operator?

An operator is formally defined as *a Kubernetes extension that uses custom resources to manage applications and their components*. More concretely, an operator encodes domain-specific operational knowledge — installation sequences, upgrade migrations, backup procedures, failure recovery — into software that runs inside the cluster.

An operator combines three things:

1. One or more **Custom Resource Definitions** (CRDs) that let users declare the desired state of an application in Kubernetes-native YAML.
2. A **controller** that watches those custom resources and continuously reconciles the real world with the declared intent.
3. **Domain knowledge** — the operational runbook that a human expert would otherwise execute by hand.

The distinction between a plain controller and an operator is precisely that domain knowledge. A controller that creates a Pod when a custom resource appears is a simple controller; one that also knows how to perform a zero-downtime database migration during an upgrade is an operator.

Operator Capability Model

A five-level maturity model describes what an operator can do at each stage. Teams use the model to scope an operator's initial release and to plan future investment.

Level	Capabilities
Level 1 — Basic Install	Automated application provisioning and configuration
Level 2 — Seamless Upgrades	Patch and minor version upgrades, including dependency updates and custom migration steps

Level	Capabilities
Level 3 — Full Lifecycle	Backup, restore, and failure recovery so the application can survive data-loss scenarios
Level 4 — Deep Insights	Metrics, alerts, log processing, and workload analysis surfaced through the operator's status
Level 5 — Auto Pilot	Horizontal and vertical scaling, auto-configuration tuning, and anomaly detection with no human input

Most open-source operators ship at Level 1 or 2. Reaching Level 3 typically requires deep knowledge of the managed application's storage and consistency model. Levels 4 and 5 add observability and closed-loop automation — capabilities that justify the operator's existence for complex stateful systems like databases, message brokers, and monitoring stacks.

Operator Frameworks

Several frameworks reduce the boilerplate of building an operator. Each makes different trade-offs between language choice, abstraction level, and ecosystem integration.

Kubebuilder is the upstream Go-based framework maintained by the Kubernetes project. It scaffolds CRDs and controllers on top of `controller-runtime`, producing the same Manager / Reconciler architecture we explored in Week 2.1. Kubebuilder projects follow a convention-over-configuration approach: after `kubebuilder init` and `kubebuilder create api`, developers fill in the `Reconcile` method and the CRD spec struct.

Operator SDK builds on Kubebuilder and adds integration with the Operator Lifecycle Manager (OLM) for packaging, dependency resolution, and catalog publishing. Beyond Go, the SDK supports two additional project types:

- **Ansible-based operators** — reconciliation logic lives in Ansible playbooks and roles, invoked automatically when a custom resource changes.
- **Helm-based operators** — each custom resource maps to a Helm chart release; the SDK renders and applies the chart on every reconciliation.

Kopf (Kubernetes Operator Pythonic Framework) lets developers write operators in Python using decorators. A handler decorated with `@kopf.on.create` fires when a custom resource is created; `@kopf.timer` sets up periodic reconciliation. Kopf is a good fit for teams that already maintain Python services and want ad-hoc operators without learning Go.

Metacontroller takes a different approach entirely. It runs as a single controller in the cluster and delegates reconciliation to user-supplied webhooks. You define a `CompositeController` or `DecoratorController` custom resource that tells Metacontroller which parent and child resources to watch, then implement a sync webhook in any language. No custom binary needs to be compiled or deployed — the webhook can even run as a serverless function.

Design Patterns

The community has converged on several patterns for keeping operators focused and composable.

One CRD per Controller

Each controller should handle exactly one custom resource type. This keeps the reconciliation loop simple, makes unit testing straightforward, and lets the Manager restart individual controllers independently. When an operator project grows to manage multiple CRDs, each CRD gets its own controller registered with the same Manager.

Single Type per Operator

An operator should manage a single type of application. Applied to a microservice stack with a web frontend, an API gateway, and a database, this pattern yields three separate operators rather than one monolithic one. The result is better separation of concerns: each operator can be versioned, upgraded, and tested on its own schedule.

Operator of Operators

When a higher-level platform needs to coordinate multiple applications — each with its own operator — a *meta-operator* manages the lifecycle of the subordinate operators. The user interacts with a single top-level CRD that represents the entire stack. The meta-operator translates that intent into custom resources consumed by the per-component operators, then aggregates their status back into the top-level resource.

This pattern appears in production systems like the IBM Cloud Pak operators, where a single `OperandRequest` CRD triggers the installation and configuration of dozens of subordinate operators. The key rule is that the meta-operator should *delegate* work to the component operators rather than duplicating their logic.

When choosing a framework, start with the language your team already knows. If you need OLM integration for catalog publishing and dependency management, Operator SDK is the natural choice. If you want to avoid compiling a binary entirely, Metacontroller's webhook model removes that step.

References

- CNCF Operator White Paper

Alternative Control Planes and Form Factors

In a standard Kubernetes cluster the control plane — `kube-apiserver`, `etcd`, `kube-controller-manager`, and `kube-scheduler` — runs alongside the workload nodes, either on dedicated machines or as static pods on control-plane nodes. That model works well for general-purpose clusters, but it is not the only option. As Kubernetes expands into multi-tenant platforms, edge computing, and resource-constrained devices, you will encounter alternative architectures that trade off different properties: management overhead, resource footprint, provisioning speed, and operational simplicity.

HyperShift

HyperShift implements *hosted control planes*: the control plane of each tenant cluster runs as a set of pods inside a central management cluster, while the tenant's worker nodes form a separate

workload cluster that joins over the network.

The management cluster treats each hosted control plane like any other workload — scheduled by the management-cluster scheduler, monitored by its observability stack, and scaled by its autoscaler. The tenant cluster's worker nodes run only the kubelet and kube-proxy; they connect back to the hosted API server through a secure tunnel.

Benefits of this architecture:

- **Faster provisioning** — creating a new cluster means deploying a set of pods rather than bootstrapping dedicated machines for the control plane. Clusters can be ready in seconds rather than minutes.
- **Reduced per-cluster overhead** — control-plane nodes are shared across tenants, eliminating the three-machine minimum that a self-hosted etcd quorum typically requires.
- **Centralized management** — upgrades, patches, and certificate rotations happen in one place for all hosted control planes.

HyperShift is the foundation of Red Hat OpenShift's hosted control plane offerings, including ROSA with hosted control planes on AWS and HyperShift on bare metal.

kcp

kcp is a Kubernetes-like control plane that serves as a generic API platform *without* scheduling workloads. It provides the Kubernetes API machinery — CRDs, controllers, RBAC, admission webhooks — but has no concept of nodes, pods, or kubelets.

kcp introduces several concepts that go beyond a standard API server:

- **Workspaces** — lightweight, isolated API scopes within a single kcp instance. Each workspace behaves like an independent Kubernetes API server, enabling massive multi-tenancy without spinning up separate clusters.
- **Transparent multi-cluster scheduling** — a *syncer* agent running in each physical cluster watches resources in kcp workspaces and creates the corresponding objects on the target cluster. Users interact with kcp as if it were a single cluster; the syncer distributes work transparently.
- **API export and binding** — workspace owners can export their CRDs as reusable APIs that other workspaces bind to, enabling a service-provider model where platform teams publish APIs consumed by application teams.

The result is a control plane that separates the *what* (API resources and their desired state) from the *where* (physical clusters that run the actual workloads).

MicroShift

MicroShift is a minimal Kubernetes distribution from Red Hat designed for edge and resource-constrained environments. It packages the API server, controller manager, scheduler, and an embedded etcd into a single binary that runs as a systemd service on RHEL-based hosts.

Key characteristics:

- **Minimal footprint** — targets devices with as little as 2 CPU cores and 2 GB of RAM.
- **OpenShift API compatibility** — exposes a subset of the OpenShift Container Platform APIs, so workloads and security policies developed for full OpenShift clusters deploy without modification.
- **Standalone operation** — designed for locations with intermittent or no connectivity to a central management hub. Updates are delivered through `rpm-ostree` and applied atomically.
- **Integrated networking** — ships with OVN-Kubernetes as its CNI plugin, providing the same network policy model as full OpenShift.

MicroShift is aimed at industrial IoT gateways, retail point-of-sale systems, and automotive platforms where a full control plane is too heavy but the OpenShift operational model is required.

k3s

k3s is a lightweight, CNCF-certified Kubernetes distribution originally created by Rancher and now maintained by SUSE. It bundles all control-plane components and the kubelet into a single binary of roughly 70 MB.

Key characteristics:

- **SQLite by default** — k3s replaces etcd with SQLite for single-server deployments, significantly reducing memory overhead. External datastores (PostgreSQL, MySQL, etcd) are supported for high-availability setups.
- **Stripped-down components** — legacy cloud-provider plugins, in-tree storage drivers, and alpha features are removed from the upstream Kubernetes source before compilation, keeping the binary small.
- **ARM support** — first-class support for ARM64 and ARMv7 makes k3s a natural fit for Raspberry Pi and industrial ARM hardware.
- **Batteries included** — k3s ships with Traefik as an ingress controller, CoreDNS, and a local-path storage provisioner out of the box. A single `curl | sh` command installs a working cluster.

k3s targets edge computing, IoT fleets, CI/CD environments, and developer workstations where a full `kubeadm`-bootstrapped cluster would be overkill.

Comparison

The following table summarises how these architectures differ from standard Kubernetes.

Aspect	Standard K8s	HyperShift	MicroShift	k3s
Control plane	Self-hosted on dedicated nodes	Hosted as pods in a management cluster	Single binary (systemd service)	Single binary

Aspect	Standard K8s	HyperShift	MicroShift	k3s
Storage backend	etcd (external or stacked)	etcd (hosted in management cluster)	etcd (embedded)	SQLite (default), etcd optional
Target environment	General purpose	Multi-tenant platforms at scale	Edge and constrained devices	Edge, dev, IoT, CI
Minimum resources	3 control-plane nodes typical	Management cluster handles overhead	2 CPU cores, 2 GB RAM	1 CPU core, 512 MB RAM

These architectures are not mutually exclusive. A platform team might run HyperShift in the data centre to manage hundreds of tenant clusters, while deploying MicroShift or k3s on edge devices that those tenants own. The choice depends on where the control plane lives, how many clusters you need, and what resources are available at each location.

References

- HyperShift
- kcp
- MicroShift: OpenShift Portability

Exercises

These hands-on labs reinforce the concepts covered in the weekly modules. Each exercise builds a small but complete Go program that you can run against a local Kubernetes cluster.

- Exercise 1: Build a Controller — Wire up the client-go pipeline from scratch
- Exercise 2: Leader Election — Add leader election to your controller
- Exercise 3: Validating Webhook — Build and deploy a validating admission webhook

Exercise 1: Build a Controller

In this exercise you will build a Kubernetes controller from scratch using client-go. By the end you will have a working program that watches for Pod events, processes them through a workqueue, and logs their state — implementing the full data flow pipeline described in client-go Architecture.

Prerequisites

- Go 1.22+ installed
- Access to a Kubernetes cluster (Kind, Minikube, or OpenShift Local)
- `kubeconfig` configured to point to your cluster

Part 1: Project Setup

Create your project directory and pull in the required dependencies:

```
mkdir k8s-controller-lab && cd k8s-controller-lab
go mod init k8s-controller-lab
go get k8s.io/client-go@latest
go get k8s.io/apimachinery@latest
```

Part 2: Connect to the Cluster

Use the standard `kubeconfig` lookup to build a REST config and create a typed clientset:

```
package main

import (
    "flag"
    "fmt"
    "path/filepath"
    "time"

    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/tools/clientcmd"
    "k8s.io/client-go/util/homedir"
```

```
)

func main() {
    var kubeconfig *string
    if home := homedir.HomeDir(); home != "" {
        kubeconfig = flag.String("kubeconfig",
            filepath.Join(home, ".kube", "config"), "path to kubeconfig")
    }
    flag.Parse()

    config, err := clientcmd.BuildConfigFromFlags("", *kubeconfig)
    if err != nil {
        panic(err)
    }
    clientset, err := kubernetes.NewForConfig(config)
    if err != nil {
        panic(err)
    }
    // ... controller setup continues below
}
```

Part 3: Set Up the Informer

Create a `SharedInformerFactory` and register event handlers that enqueue object keys. Review Informers, Listers, and Workqueues for why we enqueue keys rather than processing directly in the handler.

```
factory := informers.NewSharedInformerFactory(clientset, 10*time.Minute)
podInformer := factory.Core().V1().Pods().Informer()

queue := workqueue.NewNamedRateLimitingQueue(
    workqueue.DefaultControllerRateLimiter(), "pods")

podInformer.AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc: func(obj interface{}) {
        key, err := cache.MetaNamespaceKeyFunc(obj)
        if err == nil {
            queue.Add(key)
        }
    },
    UpdateFunc: func(old, new interface{}) {
        key, err := cache.MetaNamespaceKeyFunc(new)
        if err == nil {
            queue.Add(key)
        }
    },
    DeleteFunc: func(obj interface{}) {
        key, err := cache.DeletionHandlingMetaNamespaceKeyFunc(obj)
        if err == nil {
```

```
            queue.Add(key)
        }
    },
})

stopCh := make(chan struct{})
defer close(stopCh)
factory.Start(stopCh)
cache.WaitForCacheSync(stopCh, podInformer.HasSynced)
```

Always call `cache.WaitForCacheSync` before processing items. Starting your reconcile loop before the cache is warm leads to missed resources.

Part 4: The Worker Loop

Implement a worker that dequeues keys and processes them. On failure, use `queue.AddRateLimited(key)` to requeue with exponential backoff. On success, call `queue.Forget(key)` to reset the backoff counter.

```
func processNextItem(queue workqueue.RateLimitingInterface,
    lister corelisters.PodLister) bool {
    key, shutdown := queue.Get()
    if shutdown {
        return false
    }
    defer queue.Done(key)

    namespace, name, err := cache.SplitMetaNamespaceKey(key.(string))
    if err != nil {
        queue.Forget(key)
        return true
    }

    pod, err := lister.Pods(namespace).Get(name)
    if err != nil {
        queue.AddRateLimited(key)
        return true
    }

    fmt.Printf("Reconciling Pod %s/%s (phase: %s)\n",
        pod.Namespace, pod.Name, pod.Status.Phase)
    queue.Forget(key)
    return true
}
```

Run the worker in a goroutine and block until `stopCh` closes.

Challenge

Extend your reconcile logic: if the Pod has no labels, log a warning `"Pod {namespace}/{name} is missing required labels!"`. Then try running two instances of your controller simultaneously and observe what happens — this motivates the leader election exercise in Exercise 2.

References

- Operator SDK Go Tutorial
- Controller Rough Structure

Exercise 2: Leader Election

In this exercise you will add leader election to your controller so that only one replica is active at a time. This builds on the concepts from Leader Election in Kubernetes and Distributed Locking Theory.

Prerequisites

- The working controller from Exercise 1
- `k8s.io/client-go/tools/leaderelection` and `k8s.io/client-go/tools/leaderelection/resourcelock` imported

Step 1: Define a Unique Identity

Each controller replica needs a unique identity. In a real Deployment, use the Pod name from the downward API. For local testing, the process ID works:

```
import (
    "fmt"
    "os"

    rl "k8s.io/client-go/tools/leaderelection/resourcelock"
)

identity := fmt.Sprintf("%d", os.Getpid())
```

Step 2: Create the Lease Lock

Define a `LeaseLock` that specifies which Lease object to use for coordination:

```
lock := &rl.LeaseLock{
    LeaseMeta: metav1.ObjectMeta{
        Name:      "my-controller-lock",
        Namespace: "default",
    },
    Client: clientset.CoordinationV1(),
```

```
    LockConfig: rl.ResourceLockConfig{
        Identity: identity,
    },
}
```

Step 3: Wire Up the Election Loop

The `leaderelection` package provides a callback-based framework. Your controller's `Run` method goes inside `OnStartedLeading`:

```
leaderelection.RunOrDie(ctx, leaderelection.LeaderElectionConfig{
    Lock:            lock,
    ReleaseOnCancel: true,
    LeaseDuration:   15 * time.Second,
    RenewDeadline:   10 * time.Second,
    RetryPeriod:     2 * time.Second,
    Callbacks: leaderelection.LeaderCallbacks{
        OnStartedLeading: func(ctx context.Context) {
            runController(ctx) // your controller logic
        },
        OnStoppedLeading: func() {
            log.Println("Lost leadership, shutting down")
            os.Exit(0)
        },
        OnNewLeader: func(id string) {
            log.Printf("Current leader: %s", id)
        },
    },
})
```

The timing parameters matter. `LeaseDuration` must be greater than `RenewDeadline`, which must be greater than `RetryPeriod`. Setting `LeaseDuration` too short causes flapping during network jitter; too long delays failover.

Step 4: Test Failover

1. Run two instances of your controller in separate terminals.
2. Observe that only one prints `"I am the leader!"` and begins reconciling.
3. Kill the leader process and wait approximately 15 seconds.
4. The standby instance should acquire the Lease and start reconciling.

Verify the Lease state at any time:

```
kubectl get lease my-controller-lock -o yaml
```

The `holderIdentity` field shows which instance currently holds the lock. The `leaseTransitions` counter increments with each leadership change.

Step 5: Required RBAC

Your controller's ServiceAccount needs permission to operate on the Lease:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: leader-election
  namespace: default
rules:
- apiGroups: ["coordination.k8s.io"]
  resources: ["leases"]
  verbs: ["get", "create", "update"]
```

A Note on Fencing

The `leaderelection` package does not guarantee single-leader semantics under all conditions. If a leader pauses long enough for its Lease to expire, a new leader is elected — but the old leader may resume and briefly believe it is still leading. Kubernetes mitigates this through optimistic concurrency: updates with a stale `resourceVersion` are rejected. For a deeper discussion, see Distributed Locking Theory.

Exercise 3: Validating Webhook

In this exercise you will build and deploy a validating admission webhook that rejects Pods referencing unauthorized container registries. This applies the concepts from Admission Control Webhooks.

Prerequisites

- A running Kubernetes cluster (Kind or Minikube)
- cert-manager installed (for webhook TLS certificates)
- Go 1.22+ installed

Step 1: Implement the Webhook Handler

Create an HTTP server that receives an `AdmissionReview` request, inspects the Pod's container images, and returns an allow or deny response:

```
func handleValidation(w http.ResponseWriter, r *http.Request) {
    var review admissionv1.AdmissionReview
    if err := json.NewDecoder(r.Body).Decode(&review); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
```

```
        return
    }

    var pod corev1.Pod
    json.Unmarshal(review.Request.Object.Raw, &pod)

    allowed := true
    var message string
    for _, c := range pod.Spec.Containers {
        if !isImageAllowed(c.Image) {
            allowed = false
            message = fmt.Sprintf("image %q is not from an approved registry",
c.Image)
            break
        }
    }

    review.Response = &admissionv1.AdmissionResponse{
        UID:     review.Request.UID,
        Allowed: allowed,
        Result:  &metav1.Status{Message: message},
    }
    json.NewEncoder(w).Encode(review)
}
```

Step 2: Registry Allow-List

Accept a list of approved registry prefixes via a command-line flag:

```
var allowedRegistries []string

func init() {
    flag.Func("allowed-registries",
        "Comma-separated list of approved registries", func(s string) error {
        allowedRegistries = strings.Split(s, ",")
        return nil
    })
}

func isImageAllowed(image string) bool {
    for _, prefix := range allowedRegistries {
        if strings.HasPrefix(image, prefix) {
            return true
        }
    }
    return false
}
```

Step 3: Register the Webhook

Create a `ValidatingWebhookConfiguration` that tells the API server to send Pod CREATE and UPDATE requests to your webhook:

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: registry-validator
webhooks:
- name: validate.registry.example.com
  rules:
  - operations: ["CREATE", "UPDATE"]
    apiGroups: [""]
    apiVersions: ["v1"]
    resources: ["pods"]
  clientConfig:
    service:
      name: webhook-service
      namespace: default
      path: "/validate"
  admissionReviewVersions: ["v1"]
  sideEffects: None
  failurePolicy: Fail
```

Set `failurePolicy: Fail` in production so that Pods are blocked when the webhook is unreachable. Using `Ignore` silently skips validation, which defeats the purpose of the policy.

Step 4: Deploy and Test

1. Build a container image for your webhook and push it to your cluster's registry.
2. Create a `Deployment` and `Service` for the webhook in the `default` namespace.
3. Ensure cert-manager injects TLS certificates (the API server requires HTTPS to reach webhooks).
4. Apply the `ValidatingWebhookConfiguration`.

Test with an unauthorized image:

```
kubectl run test --image=docker.io/library/nginx:latest
# Expected: Error from server: admission webhook
#   "validate.registry.example.com" denied the request:
#   image "docker.io/library/nginx:latest" is not from an approved registry
```

Test with an authorized image (assuming `quay.io` is in your allow-list):

```
kubectl run test --image=quay.io/myorg/myapp:latest
# Expected: pod/test created
```

Tips

- Use `kubectl run --dry-run=server` during development to test webhook responses without creating actual Pods.
- Check webhook logs if requests are being denied unexpectedly — the `AdmissionReview` request object contains the full Pod spec.
- For a declarative alternative to hand-coded webhooks, see Policy Engines and Gatekeeper.

Recommended Reading

The following resources are strongly recommended as companions to this course. Each book or paper aligns with specific weekly modules and will deepen your understanding of the topics covered.

Programming Kubernetes

Michael Hausenblas and Stefan Schimanski (O'Reilly, 2019)

This book is the definitive guide to client-go internals, custom resources, and the API machinery that powers Kubernetes extensibility. It serves as an excellent companion to Week 1.1 and Week 1.2, where you will explore the Kubernetes resource model and controller foundations. Readers will gain a thorough understanding of how the API server processes requests and how to build robust clients and controllers.

Kubernetes Operators

Jason Dobies and Joshua Wood (O'Reilly, 2020)

This book covers the operator pattern end-to-end, from the Operator SDK to lifecycle management of complex applications on Kubernetes. It pairs well with Week 2.1 and Week 3.1, where you will write controllers with controller-runtime and extend the API server. It provides practical guidance on packaging, deploying, and upgrading operators in production.

CNCF Operator White Paper

CNCF TAG App Delivery — Operator White Paper

This industry reference defines the operator capability model and provides a comprehensive comparison of operator frameworks. It is the ideal companion to Week 4.2, where you will survey the operator landscape and alternative control plane architectures. The white paper offers a vendor-neutral perspective on when and how to build operators. # Recommended Reading

Recommended Reading

To deepen your understanding of Kubernetes development and distributed systems, the following resources are recommended:

- **Core Kubernetes Development:**
  - **Programming Kubernetes** by Michael Hausenblas and Stefan Schimanski: Covers the fundamentals of extending Kubernetes.
  - **Kubernetes Operators** by Jason Dobies and Joshua Wood: Essential for understanding automation and controller design.
  - **CNCF Operator Whitepaper**: Provides standardized guidance on the operator pattern.

- **Security and System Architecture:**
  - **Kubernetes Security and Observability** by Brendan Creane and Amit Gupta: Best practices for securing and monitoring clusters.
  - **Designing Distributed Systems** by Brendan Burns: Explores reusable patterns for building scalable, reliable applications on Kubernetes.

Recommended Reading

Programming Kubernetes (Hausenblas, Schimanski)

Recommended Reading: Programming Kubernetes

In the journey toward becoming a Kubernetes developer, understanding the "how" and "why" of the control plane is essential. Michael Hausenblas and Stefan Schimanski's **Programming Kubernetes: Developing Cloud-Native Applications** serves as the definitive foundational text for mastering the Kubernetes API and extending the platform.

Why this book is essential

While many resources focus on using `kubectl` to manage workloads, this text shifts the perspective to that of a **platform engineer**. It deconstructs the Kubernetes control plane, providing the technical depth required to transition from a consumer of Kubernetes resources to an architect of Kubernetes extensions.

Key insights provided by the authors include:

- **API-Centric Architecture:** A deep dive into how the `kube-apiserver` acts as the single source of truth and the gateway for all declarative configuration.
- **The Controller Pattern:** A clear explanation of the asynchronous reconciliation loop, which is the mechanism that maintains the desired state of the cluster.
- **Deep Dive into `client-go`:** Essential guidance on using the official Go client library to interact with the Kubernetes API, manage informers, and handle event propagation.
- **Custom Resource Definitions (CRDs):** Practical strategies for extending the Kubernetes API to manage domain-specific resources.

Key Concepts Covered

For developers following this learning path, the book aligns perfectly with our core objectives:

Topic	Relevance to Learning Path
The Control Plane	Provides the theoretical backing for our study of `etcd`, API validation, and the reconciliation pattern.
Custom Controllers	Mirrors our focus on building operators that utilize `client-go`, workqueues, and cache mechanisms.
Distributed Systems	Offers guidance on implementing the eventual consistency model and handling concurrency—crucial for high-availability operators.

How to use this resource in this course

We recommend using **Programming Kubernetes** as your primary reference manual during the hands-on labs. Specifically, as you begin building your first controller from scratch, refer to the

chapters covering:

1. **Workqueues and Informers:** Use these sections to understand the flow of events from the API server to your custom controller logic.
2. **API Machinery:** Use this as a reference when defining your Custom Resource Definition (CRD) schemas and managing `Spec` vs. `Status` fields.
3. **Optimistic Concurrency:** When we tackle the lesson on "Going distributed: locks, leases, and ownership," the book's coverage of resource versioning and conflict handling will be invaluable for implementing robust leader election.

Further Study

This book is not a "read once" resource; it is an architectural handbook. As you progress from basic controllers to advanced admission webhooks and custom API servers, revisit the technical implementation details documented in the text to ensure your custom infrastructure components adhere to the standard Kubernetes "look and feel."

For those currently navigating the transition from `kubectl` automation to binary-based controller development, focus heavily on the sections describing the `client-go` event-driven architecture. This is the cornerstone of the hands-on labs you will complete in this track.

Kubernetes Operators (Dobies, Wood)

Kubernetes Operators: Automating the Orchestration Platform

Understanding the operational paradigm of Kubernetes requires moving beyond simple deployments toward the concept of **Operators**. As defined by Jason Dobies and Joshua Wood in **Kubernetes Operators: Automating the Container Orchestration Platform**, an Operator is essentially a software extension to Kubernetes that makes use of custom resources to manage applications and their components.

The Operator Pattern

At its core, the Operator pattern extends the Kubernetes control plane's capabilities to manage complex, stateful applications—such as databases, monitoring stacks, or message brokers—that would otherwise require manual intervention.

The pattern relies on the **Reconciliation Loop**, where the controller: 1. **Observes:** Watches the API server for changes to the Custom Resource (CR). 2. **Analyzes:** Compares the "Desired State" (the Spec defined in the CR) with the "Observed State" (the current status of the cluster). 3. **Acts:** Executes the necessary CRUD operations (Create, Update, Patch, Delete) to align the observed state with the desired state.

Key Takeaways from Dobies & Wood

The text provides a architectural framework for moving from a standard controller to a fully functional Operator:

- **Custom Resources (CRs):** Operators leverage Custom Resource Definitions (CRDs) to expose an API that represents your domain-specific configuration.
- **The Reconcile Logic:** Unlike a standard loop, an Operator must be idempotent. If a process is interrupted or a component is partially deployed, the next iteration of the loop should safely resume or correct the state without causing side effects.
- **Spec vs. Status:** A critical distinction in the Operator lifecycle. The `Spec` represents the configuration requested by the user, while the `Status` represents the current reality of the application. Effective Operators report rich, actionable data in the `Status` field so users can monitor progress.

Comparison: Kubebuilder vs. Operator SDK

When beginning your development journey, choosing the right toolset is vital. While **Kubernetes Operators** focuses on the conceptual underpinnings, practical implementation often leans on the **Operator SDK** or **Kubebuilder**.

| Feature | Kubebuilder | Operator SDK | | :--- | :--- | :--- | | **Philosophy** | "Go-native," focuses on simplicity and standard `controller-runtime` patterns. | Built on top of Kubebuilder; provides additional "batteries-included" features. | | **Capability Levels** | Primarily Go-based. | Supports Go, Helm, and Ansible-based operators. | | **Tooling** | Minimalist; standard scaffold. | Includes features like `operator-scorecard` and enhanced lifecycle management. |

Recommended Practical Workflow

To successfully implement the concepts discussed by Dobies and Wood, align your development process with these industry standards:

1. **Define your API:** Before writing logic, follow the Kubernetes API Conventions.
2. **Lint your API:** Use `kube-api-linter` integrated with `golangci-lint` to catch common mistakes in your CRD definitions (e.g., missing validation or incorrect field types).
3. **Choose your Abstraction:** If you are building high-performance, complex systems, prioritize `controller-runtime` (via Kubebuilder) to maintain fine-grained control over the reconcile loop.
4. **Manage State:** Always prefer `PATCH` or `UPDATE` semantics that account for optimistic concurrency. Use standard Kubernetes `ResourceVersion` checks to prevent race conditions during updates.

Further Learning

For deep dives into the topics covered, refer to these resources: * **Kubernetes Operators** by Dobies and Wood (O'Reilly). * CNCF Operator Whitepaper: Provides a comprehensive view of how Operators fit into the broader cloud-native ecosystem. * The Kubebuilder Book: Essential documentation for mastering the reconciliation pattern in Go.

CNCF Operator Whitepaper

CNCF Operator Whitepaper

The CNCF Operator Whitepaper serves as the foundational architectural guide for developers

looking to extend Kubernetes through automation. As you transition from managing individual resources to building autonomous systems, understanding the principles defined in this document is essential for creating resilient, production-grade Operators.

The Essence of an Operator

An Operator is a software extension to Kubernetes that makes use of custom resources to manage applications and their components. Following the CNCF definitions, an Operator implements the **Control Loop**—a fundamental pattern in distributed systems that ensures the current state of a system matches the desired state defined by the user.

Key Components of an Operator

According to the CNCF whitepaper, an effective Operator relies on three primary pillars:

1. **Custom Resource Definitions (CRDs):** These allow you to define your own API objects, extending the Kubernetes API to understand the domain-specific logic of your application (e.g., a `Database` or `Queue` resource).
2. **Controller Logic:** The brain of the operation. It continuously watches the cluster state, compares it against the declared intent, and performs actions (CRUD operations) to reconcile discrepancies.
3. **Human Operational Knowledge:** An Operator is only as good as the operational experience it encodes. This includes lifecycle management tasks like installation, upgrades, backups, and recovery.

The Operator Maturity Model

A core contribution of the CNCF Operator whitepaper is the **Operator Maturity Model**. When designing your controller, it is helpful to categorize your goals into these levels:

- **Phase I: Basic Install:** The Operator automates the deployment of the application (e.g., creating Deployments, Services, and ConfigMaps).
- **Phase II: Seamless Upgrades:** The Operator handles application version updates and patch management.
- **Phase III: Full Lifecycle:** The Operator manages storage, backups, and scaling (e.g., handling persistent volume snapshots).
- **Phase IV: Deep Insights:** The Operator provides observability metrics, alerting, and log analysis.
- **Phase V: Auto-pilot:** The Operator performs automated tuning, horizontal scaling, and self-healing without human intervention.

Architectural Best Practices

When building your controllers using `client-go` or `controller-runtime`, refer to these whitepaper-recommended practices:

- **Idempotency:** Every action performed by the reconciliation loop must be idempotent. If an error occurs, the controller will requeue the work item; your code must be able to handle

"seeing" the same state multiple times without causing side effects.

- **Decoupled Logic:** Separate your API definition (the "Spec") from the operational implementation. This allows you to evolve your API surface independently of your controller's internal reconciliation logic.
- **Observability:** Operators act as "privileged" actors in the cluster. Always expose metrics (via Prometheus) so that users can monitor the health and progress of your control loop.

Further Reading

To deepen your understanding of these patterns, the CNCF documentation provides critical insights into how Operators interact with the Kubernetes API machinery:

- **CNCF Tag App Delivery:** Visit the [official CNCF Operator Whitepaper](https://tag-app-delivery.cncf.io/whitepapers/operator/) for the complete architectural specification.
- **Designing Distributed Systems:** Complement your learning by reading **Designing Distributed Systems** by Brendan Burns, which covers the sidecar and controller patterns that underpin the Operator framework.

When implementing your first `client-go` controller, keep in mind that the **WorkQueue** is your buffer. Informers watch for changes, Listers read from the cache, and the WorkQueue ensures that your reconciliation logic processes these events in a FIFO (First-In-First-Out) manner. Never attempt to bypass the Informer cache to read directly from the `kube-apiserver`, as this introduces latency and unnecessary load on the control plane.

Suggested Reading

The following resources provide additional depth on topics covered in this course. While not required, they are valuable references that complement specific weekly modules.

Kubernetes Security and Observability

Brendan Creane and Amit Gupta (O'Reilly, 2021)

This book is a practical guide to implementing robust security postures and effective monitoring strategies in Kubernetes clusters. It covers network policies, RBAC design, runtime security, and observability patterns in detail. It serves as an excellent companion to Week 3.2, where you will study networking, authentication, authorization, and cluster hardening.

Designing Distributed Systems

Brendan Burns (O'Reilly, 2024)

This book explores common distributed systems patterns — sidecar, ambassador, adapter — and shows how they are implemented with Kubernetes. It provides a strong conceptual foundation for understanding the distributed challenges covered in Week 2.2, including idempotency, leader election, and controller sharding. # Suggested Reading

Suggested Reading

The recommended literature for the Kubernetes Developer Learning Path focuses on securing, observing, and designing scalable distributed applications:

- **Kubernetes Security and Observability** (Creane & Gupta): Provides essential strategies for hardening cluster security and implementing effective observability practices.
- **Designing Distributed Systems** (Burns): Explores common, reusable software patterns designed to build scalable and reliable systems using Kubernetes primitives.

Suggested Reading

Kubernetes Security and Observability (Creane, Gupta)

Kubernetes Security and Observability

As Kubernetes clusters move from experimental projects to production-grade infrastructure, the complexity of securing and monitoring the environment grows exponentially. This module explores the core concepts of Kubernetes security and observability, drawing from the architectural insights provided by Brendan Creane and Amit Gupta in **Kubernetes Security and Observability**.

Security Foundations in Kubernetes

Securing a Kubernetes environment requires a defense-in-depth approach. Unlike traditional virtualized environments, Kubernetes requires security controls to be applied at multiple layers: the container image, the node, the network, and the API server.

The Security Architecture

The `kube-apiserver` acts as the central gatekeeper. As noted in the resource model, every request undergoes a strict lifecycle: * **Authentication:** Verifying who is making the request (X.509, OIDC, Tokens). * **Authorization:** Determining if the identity has the permissions for the requested action (RBAC, Node Authorization, ABAC). * **Admission Control:** Validating or mutating the request before it is persisted in `etcd`.

Policy Enforcement

A critical aspect of securing the cluster is moving beyond native RBAC toward automated governance. This involves: * **Validating Webhooks:** Intercepting requests to ensure they comply with organizational standards. * **Policy Engines:** Utilizing tools like OPA Gatekeeper or Kyverno to decouple security policies from application code, allowing for centralized, declarative governance.

Observability: Understanding Cluster State

Observability in Kubernetes is more than just logging; it is the ability to understand the internal state of the system by examining its external outputs (metrics, logs, and traces).

The Three Pillars

1. **Metrics:** Quantifiable data (e.g., CPU/Memory usage, request latency, error rates). These are typically gathered via the Metrics Server or Prometheus and provide the "what" is happening in the system.
2. **Logs:** Immutable records of discrete events (e.g., application startup, connection failures). Logs

provide the "why" something happened.

3. **Traces:** Distributed tracing allows engineers to follow a request as it traverses microservices, which is essential for troubleshooting performance bottlenecks in complex, distributed systems.

Hands-on Activity: Analyzing Security Contexts

In this exercise, we will apply security hardening principles by restricting a Pod's capabilities through the `SecurityContext`.

Create a restricted pod manifest (`restricted-pod.yaml`):

```
apiVersion: v1
kind: Pod
metadata:
  name: security-hardened-pod
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
  containers:
  - name: nginx
    image: nginx:latest
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop:
          - ALL
```

Steps to apply and verify:

1. **Apply the configuration:**

   ```
   kubectl apply -f restricted-pod.yaml
   ```

2. **Verify security constraints:** Attempt to enter the container and perform a privileged action (like changing system time) to confirm the `capabilities` block effectively dropped `ALL` privileges.

   ```
   kubectl exec -it security-hardened-pod -- date -s "2025-01-01"
   # Expected output: Operation not permitted
   ```

Troubleshooting and Best Practices

- **Least Privilege:** Always grant only the permissions necessary for a controller or user to function. Use `RoleBindings` scoped to specific namespaces rather than `ClusterRoleBindings` whenever possible.
- **Visibility:** Ensure your observability stack (Prometheus/Grafana) has access to API audit logs.

Audit logs are the single most important tool for forensic analysis in a compromised cluster.

- **Immutable Infrastructure:** Treat your nodes and containers as immutable. If a container is misbehaving, replace it rather than patching it in place.

Recommended Reading

- Creane, Brendan, and Amit Gupta. **Kubernetes Security and Observability**. O'Reilly Media, Inc., 2021.

Designing Distributed Systems (Burns)

Designing Distributed Systems: Foundations for Kubernetes Controllers

In the context of Kubernetes development, transitioning from a single-instance controller to a resilient, distributed operator requires a fundamental shift in how we perceive state. Brendan Burns' **Designing Distributed Systems** provides the theoretical bedrock for the patterns we implement in `client-go` and `controller-runtime`.

To build robust Kubernetes controllers, you must move away from "transactional" thinking—where you expect a command to succeed immediately—and move toward "eventual consistency."

Core Distributed Principles for Kubernetes

When scaling controllers or ensuring high availability, we rely on specific patterns to manage state across distributed nodes:

- **The Sidecar Pattern:** Extending the functionality of a primary container without modifying its source code. In controller design, this is mirrored in how we use `Admission Webhooks` to intercept and modify resource requests before they reach `etcd`.
- **The Ambassador Pattern:** Offloading communication logic (like retries or mTLS) to a sidecar, similar to how Service Mesh architectures (like Istio or Linkerd) handle traffic management outside the application logic.
- **The Work Queue Pattern:** Essential for `client-go` informers. By decoupling the observation of state (events) from the processing of state (the Reconcile loop), we ensure that a spike in API traffic does not crash our controller.
- **The Replicated Worker Pattern:** The foundation of controller sharding. When you run multiple instances of a controller, you must ensure they don't fight over the same resources.

Implementing Distributed State Machines

The transition from a basic controller to a distributed system is defined by how you handle conflicts and state ownership.

1. Idempotency and Atomicity

In a distributed system, you cannot assume a request will only arrive once. Your `Reconcile` function must be **idempotent**: * **Definition:** An operation is idempotent if performing it multiple times yields the same result as performing it once. * **Application:** Always verify current cluster state

before applying changes. If the resource is already in the desired state, the `Reconcile` loop should exit gracefully without making unnecessary API calls.

2. Leader Election (Leases)

Running multiple replicas of a controller for high availability (HA) introduces a "Split Brain" risk, where two controllers attempt to modify the same resource simultaneously. * **The Mechanism:** Kubernetes uses the `Lease` resource (within the `coordination.k8s.io` API group) to implement distributed locking. * **The Workflow:** 1. Controllers compete to acquire a `Lease` object. 2. The controller that holds the lock is the "Leader." 3. Other instances wait or remain in a "standby" mode. 4. If the leader fails to renew the lease, a new leader is elected.

3. Controller Sharding

When a single controller becomes a bottleneck (e.g., managing thousands of Ingress resources), you implement **sharding**. By partitioning the work—where each replica is responsible for only a subset of resources (e.g., resources with a specific label or hash)—you increase throughput while maintaining individual controller simplicity.

Hands-on Concept: The Distributed Reconcile Logic

When writing your own controller (as outlined in our 2.2 objectives), apply these design principles:

```
// Simplified pseudo-code for a Reconcile loop demonstrating distributed principles
func (r *Reconcile) Reconcile(req ctrl.Request) (ctrl.Result, error) {
    // 1. Fetch the latest state (Do not rely on cached memory)
    instance := &v1.MyResource{}
    err := r.Get(ctx, req.NamespacedName, instance)

    // 2. Idempotent logic: Check if work is actually needed
    if instance.Status.Processed == true {
        return ctrl.Result{}, nil
    }

    // 3. Perform the work
    // Ensure this call is atomic or repeatable
    err = r.performAction(instance)

    // 4. Update Status (Optimistic Locking)
    // Always use Patch/Update to ensure we don't overwrite changes made by other
controllers
    return ctrl.Result{}, err
}
```

Key Takeaways for the Developer

- **Design for Failure:** Assume the network is unreliable and the API server may reject your request. Always use exponential backoff in your workqueues.
- **Respect the State:** Kubernetes is the source of truth. Your controller is merely a tool to drive

the cluster toward a desired configuration.

- **Leverage Existing Patterns:** Don't reinvent locks; use `client-go` leaderelection packages that utilize the `Lease` API to keep your implementation clean and idiomatic.

For further exploration, **Designing Distributed Systems** by Brendan Burns is essential reading to understand the transition from standalone scripts to production-grade, distributed Kubernetes operators.

Appendix

Resources

Download Course PDF

Glossary

API Server (kube-apiserver)

The central management component of the Kubernetes control plane. It exposes the Kubernetes API, validates and configures data for API objects, and serves as the front end for the cluster's shared state.

CRD (Custom Resource Definition)

A mechanism for extending the Kubernetes API by defining new resource types without building a custom API server. CRDs allow users to create and manage custom objects using standard Kubernetes tooling.

Controller

A control loop that watches the state of a cluster through the API server and makes changes to move the current state toward the desired state. Controllers are the core automation mechanism in Kubernetes.

Controller-runtime

A set of Go libraries for building Kubernetes controllers and operators. It provides high-level abstractions over client-go, including the Reconcile loop pattern, and is used by kubebuilder.

client-go

The official Go client library for interacting with the Kubernetes API. It provides typed clients, dynamic clients, discovery, informers, listers, and workqueues.

DeltaFIFO

A queue implementation in client-go that stores deltas (changes) for Kubernetes objects. It feeds events to the Informer's event handlers in the order they were received.

etcd

A distributed, consistent key-value store used as the backing store for all Kubernetes cluster data. It provides strong consistency guarantees through the Raft consensus algorithm.

Finalizer

A key on a Kubernetes resource that prevents the resource from being deleted until the controller responsible for the finalizer has completed its cleanup logic. Finalizers enable graceful teardown of dependent resources.

GVK (GroupVersionKind)

A tuple that uniquely identifies the type of a Kubernetes API object by its API group, version, and

kind. For example, `apps/v1 Deployment` has group `apps`, version `v1`, and kind `Deployment`.

GVR (GroupVersionResource)

A tuple that uniquely identifies a Kubernetes API resource endpoint by its API group, version, and resource name. GVR is used for RESTful URL construction, while GVK is used for object type identification.

Informer

A client-go component that watches for changes to Kubernetes resources and maintains an in-memory cache. Informers reduce load on the API server by serving reads from cache and delivering events through handlers.

KAL (kube-api-linter)

A linter for Kubernetes API types that enforces API conventions and best practices. It checks Go struct definitions for CRDs against the Kubernetes API guidelines.

Lease

A lightweight Kubernetes resource used to implement distributed coordination primitives such as leader election and heartbeating. Leases allow controllers to claim and renew ownership of a coordination lock.

Leader Election

A mechanism by which multiple replicas of a controller elect a single active instance (the leader) to perform work, while the others stand by. It is implemented using Leases or ConfigMaps in Kubernetes.

Lister

A client-go component that reads Kubernetes objects from the Informer's local cache rather than making API server calls. Listers provide fast, low-latency reads for controller logic.

OCC (Optimistic Concurrency Control)

A strategy used by the Kubernetes API server to detect conflicting writes. Each resource carries a `resourceVersion` field; updates that specify a stale version are rejected, forcing the client to re-read and retry.

Operator

A pattern for packaging, deploying, and managing a Kubernetes application using custom resources and custom controllers. Operators encode domain-specific operational knowledge into software that runs on the cluster.

Reconcile Loop

The core pattern in controller-runtime where a controller receives a request for a specific object and compares the current state to the desired state, taking action to converge them. Each invocation should be idempotent.

Reflector

A client-go component that watches the Kubernetes API server for a specific resource type and pushes changes into a DeltaFIFO queue. It is the first stage of the Informer pipeline.

Scheme

A registry in the Kubernetes API machinery that maps Go types to GVKs and vice versa. The Scheme is used for serialization, deserialization, and defaulting of API objects.

Server-Side Apply (SSA)

An API server feature that tracks field ownership per manager, enabling safe concurrent updates by multiple controllers. SSA uses Apply operations instead of traditional Update or Patch, reducing merge conflicts.

SharedInformerFactory

A factory in client-go that creates and manages shared Informers for multiple resource types. Sharing Informers across controllers reduces the number of API server watch connections.

Spec

The section of a Kubernetes resource that declares the desired state. Users and controllers write to the Spec to express intent; controllers then reconcile the cluster to match.

Status

The section of a Kubernetes resource that reports the observed state as determined by controllers. Status is typically updated by the controller that manages the resource, not by end users.

Workqueue

A rate-limited, de-duplicating queue in client-go used by controllers to process events. Items are added by Informer event handlers and consumed by worker goroutines in the controller's reconcile loop.